HORIZON

Dossier tecnico

Strumento di volo per ultraleggeri e piloti VFR · app Android (Capacitor), versione web per iPhone e browser, HORIZON 2 con l'orologio Garmin, automazioni su GitHub Actions

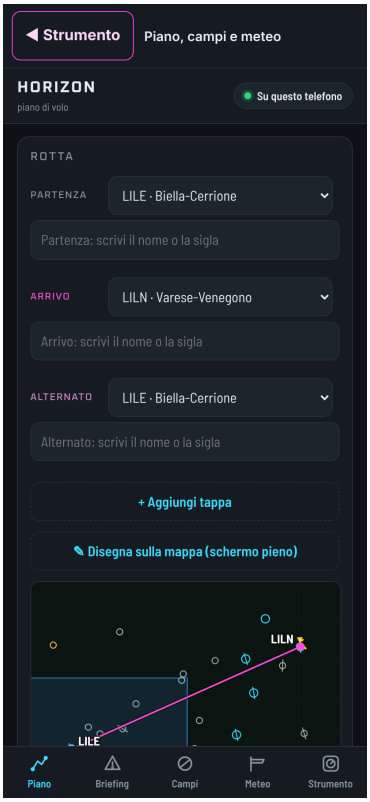
Codice del ramo principale al 10 novembre 2026 · ultima versione nelle novità dell'app: 1.0.161



Orizzonte (volo dimostrativo)



Mappa mobile (senza tessere)



Piano di volo

Documento tecnico. Descrive come è fatto il software HORIZON e come funziona, leggendo il codice del repository: architettura, sensori, schermate, dati, moduli nativi, orologio, licenze, compilazione e collaudo. Non è un manuale d'uso: quello è il «Manuale dell'app». HORIZON è uno strumento sperimentale, non certificato: il pilota in comando resta l'unico responsabile del volo.

© 2026 Enrico D'Amico · Tutti i diritti riservati · horizonlicenses@gmail.com

Indice

1. Panoramica e architettura 4
1.1 Canali di distribuzione 4
1.2 Come Capacitor impacchetta www/ 4
1.3 Il ciclo dello strumento 5
2. Mappa del repository 6
2.1 Servizi esterni chiamati dall'app 8
3. Sensori e acquisizione dei dati 9
3.1 Posizione: GPS del telefono, Garmin GLO 2, orologio 9
3.2 Assetto: accelerometro e giroscopio 11
3.3 Bussola 13
3.4 Quota, vario e QNH 14
3.5 Traffico GDL90 (SafeSky, SkyEcho, PilotAware) 15
3.6 Stato del telefono 15
4. Strumento e schermate 16
4.1 Orizzonte (PFD) 16
4.2 Mappa 16
4.3 Terreno (DEM) e TAWS 17
4.4 Avvisi 17
4.5 Vista 3D, schede, schermo diviso, voce, emergenza 17
5. Piano di volo (app.html) 19
5.1 Rotta 19
5.2 Briefing, meteo, spazi aerei, NOTAM 20
5.3 Velivolo, carburante, prestazioni, diario 20
6. Volare senza internet: cache, carta VFR, spazi aerei 21
6.1 Service worker 21
6.2 Carta VFR open flightmaps 21
6.3 Ciclo AIRAC e fonti degli spazi aerei 22
7. Moduli nativi Android (Java) 23
7.1 La variante HORIZON 2 (-Ph2) 23
8. Orologio Garmin (Connect IQ) 24
8.1 Dall'orologio al telefono 24
8.2 Dal telefono all'orologio, tasti, avvisi 25
8.3 Scatola nera e diagnosi delle chiusure 25
8.4 Compilazione (horizon2.yml) 25
9. Licenze 26
9.1 Formato del codice 26
9.2 Verifica sul telefono, revoca, menu riservato 26
9.3 Automazione per posta 27
10. Compilazione e pubblicazione (GitHub Actions) 30
10.1 Numeri di versione 30

© 2026 Enrico D'Amico · Tutti i diritti riservati

10.2 Dettagli dei workflow . . . 30
10.3 Segreti usati dai workflow . . . 31
11. Prove e qualità . . . 32
11.1 Banco di prova dell'assetto (tools/test_assetto.js) . . . 32
11.2 Registrazioni dei sensori . . . 32
11.3 Collaudo sull'emulatore . . . 32
11.4 Archivio degli scarti dei sensori . . . 32
11.5 Altre verifiche . . . 32
12. Sicurezza e privacy . . . 33
12.1 Chiavi e segreti . . . 33
12.2 Cosa è pubblico . . . 33
12.3 Impronta e filigrana dell'autore . . . 33
12.4 Privacy (sintesi di PRIVACY.md) . . . 33
13. Proposte e rischi . . . 34
14. Appendice: il codice . . . 35

© 2026 Enrico D'Amico · Tutti i diritti riservati

1. Panoramica e architettura

HORIZON porta sul telefono lo strumento da 80 mm nato per un Avid Flyer: orizzonte sintetico con terreno vero, mappa mobile, navigazione con circuito e atterraggio, avvisi di terreno, spazi aerei, tramonto e traffico, piano di volo con meteo e briefing, diario di volo. Il codice è quasi tutto JavaScript in due pagine HTML (`www/index.html`, lo strumento, e `www/app.html`, il Piano) più una ventina di moduli in `www/`; la parte Android aggiunge pochi moduli nativi in Java per quello che il browser non sa fare (Bluetooth seriale, bussola di Android, UDP, carta VFR in SQLite, firma dell'app e Play Billing, aggiornamenti, stato del telefono).

Lo stesso codice gira in tre forme: come app Android (Capacitor impacchetta `www/` dentro un'APK), come sito su GitHub Pages (PWA, da aggiungere alla schermata Home su iPhone) e come «HORIZON 2», un'app Android a parte con il collegamento all'orologio Garmin. Le differenze tra le versioni le decide `www/version.js`, scritto dai workflow con il campo `channel` (`apk`, `play`, `web`, `h2`).

1.1 Canali di distribuzione

Canale	Come arriva al pilota	channel	Cosa cambia
APK da GitHub	Release «HORIZON 1.0.N» (tag `build-N`), installazione a mano sopra la precedente	apk	Permesso `REQUEST_INSTALL_PACKAGES` aggiunto in compilazione; aggiornamenti dall'app (`UpdaterPlugin.apkDownload/apkInstall`, controllo su `api.github.com/…/releases/latest`); donazione PayPal
Google Play	AAB caricato da solo nel canale «Test interni»; la Produzione si pubblica a mano	play	Niente permesso di installare pacchetti; aggiornamento con Play In-App Updates (`playCheck`, `playStart`); «Offrimi un caffè» con Play Billing (prodotto `caffe`)
Web / iPhone (PWA)	`amicoen-ops.github.io/HORIZON`, su iPhone Safari › Condividi › Aggiungi alla schermata Home	web	Nessun modulo nativo: GLO 2 con Web Serial dove c'è, carta VFR con il worker sqlite-wasm, IGM e ortofoto non disponibili (servizio solo http); aggiornamento proposto confrontando l'impronta in `version.js?chk=…`
HORIZON 2 (prova)	Release pre-release `horizon2-N`; app `it.amicoen.horizon2` che si installa accanto a HORIZON	h2	Stesso `www/` più il modulo `GarminPlugin` (compilazione `-Ph2`) e l'app «HORIZON Link» per fenix 7 / epix 2; mai su Play

1.2 Come Capacitor impacchetta www/

- `capacitor.config.json`: `webDir` = `www`, `androidScheme` = `https` (la pagina ha origine `https://localhost`), `cleartext:false`, `allowMixedContent:true` (serve solo al WMS http del Geoportale), `captureInput:true`, `webContentsDebuggingEnabled:false` (acceso solo nell'APK di collaudo per l'emulatore).
- `npx cap sync android` copia `www/` negli asset dell'app e collega i plugin npm: `@capacitor/filesystem`, `@capacitor/share`, `@capacitor-community/keep-awake`, `@capacitor-community/text-to-speech` (Capacitor 8.6.0).
- `MainActivity` registra i 7 plugin nativi del progetto e, solo se la classe esiste (HORIZON 2), `GarminPlugin` con `Class.forName`; imposta `setTextZoom(100)` (il carattere ingrandito di sistema non deve rompere lo strumento), `FLAG_KEEP_SCREEN_ON` e lo schermo intero immersivo.
- In pagina `PH.native` vale `null` nel browser; nell'app contiene i riferimenti ai plugin (`TextToSpeech`, `KeepAwake`, `Filesystem`, `Share`, `CapacitorHttp`, `GloSerial`, `Compass`, `Updater`). `IA517.http(url)` usa `CapacitorHttp` nell'app (niente limiti CORS) e `fetch` con 12 s di timeout nel browser.
- Nella versione web il workflow toglie `capacitor.js` e il suo tag `<script>`; `sw.js` riconosce l'app (host `localhost` senza porta) e non mette in cache le pagine, che sono già nell'APK.
- Origine storica: `index.html` è lo strumento del Raspberry Pi con la scheda sensori XIAO; la sezione «VERSIONE TELEFONO / TABLET» ridefinisce le funzioni (per esempio `linkStart()` diventa una funzione vuota) e `PH.handle` risponde alle stesse richieste REST che il Piano faceva al servizio del Raspberry.

© 2026 Enrico D'Amico · Tutti i diritti riservati

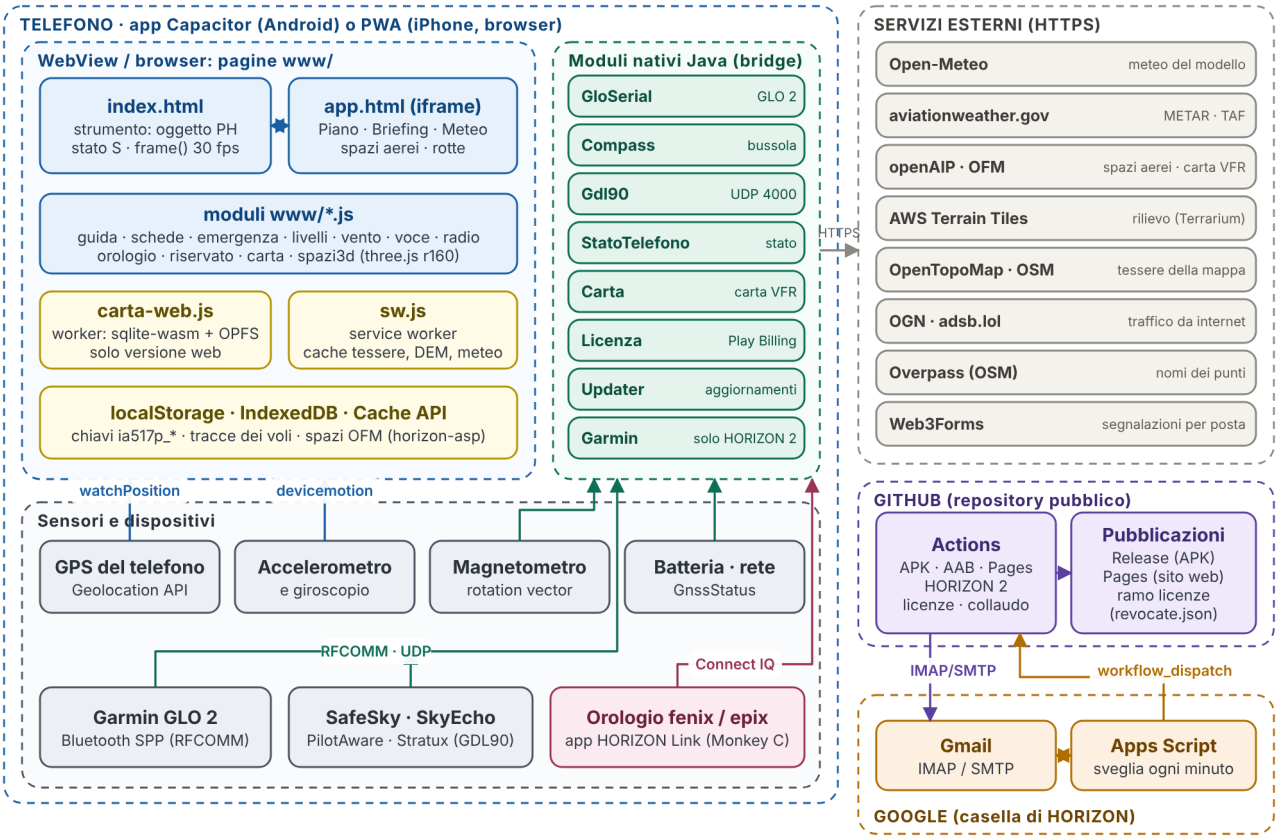


Figura 1 — Architettura dei componenti: pagine e moduli JavaScript nella WebView, moduli nativi Java, sensori e dispositivi esterni, servizi in rete, GitHub (Actions, Release, Pages, ramo «licenze») e la casella Gmail con lo script di sveglia.

1.3 Il ciclo dello strumento

Lo strumento disegna con `requestAnimationFrame(frame)`: `frame()` limita i fotogrammi a `PH.fps` (30 di serie, `ia517p_fps`), misura il tempo di disegno, registra i blocchi oltre 1,5 s e chiama `frameBody` → `drawPFD` o `drawMap`. Un errore di disegno non ferma il ciclo: il fotogramma successivo è già prenotato e a schermo compare «ERRORE DI VISUALIZZAZIONE». Prima di ogni fotogramma `PH.sane()` riporta a valori sicuri i numeri impossibili (quota oltre 60.000 ft, assetto non numerico…). `PH.afterFrame` fa il resto con tre ritmi:

Ritmo	Cosa fa
ogni fotogramma	schermo diviso del tablet (`PH.drawSplit`), traffico sulla mappa e triangolo di pericolo (`PH.trfMap`, `PH.trfDraw`)
ogni 0,5 s	caselle dei dati e riga di stato (`PH.info`), tasti dello zoom, zoom salvato
ogni 1 s	campi vicini, spazio aereo attuale (`PH.renderAsp`), simulazione (`PH.simTick`), vento previsto, avvisi di terreno, spazi aerei e tramonto, traffico (`PH.trfTick`)
ogni 5 s	stato del telefono (`PH.statoPoll`: batteria, rete, satelliti)
eventi	sensori di movimento, GPS, bussola, GLO 2, GDL90 e orologio arrivano quando arrivano e aggiornano lo stato `S`

Lo stato del volo è l'oggetto globale `S` (`pitch`, `roll`, `hdg`, `trk`, `gs`, `ias`, `alt`, `vs`, `slip`, `lat`, `lon`, `gpsReal`, `sats`, `hdop`…), esposto anche come `window.phS` per il Piano. Le scadenze sono in `STALE` (assetto 0,7 s; velocità e quota 2,5 s; GPS 3,5 s) e i tempi dell'ultimo dato in `LIVE`: se un dato scade, `drawFailFlags` copre la scala con una X rossa e una bandierina («ASSETTO NON DISPONIBILE», «GPS PERSO»): mai un valore congelato.

© 2026 Enrico D'Amico · Tutti i diritti riservati

2. Mappa del repository

Righe contate al momento della generazione di questo documento. I file vendorizzati (librerie di terze parti) sono indicati come tali e non sono descritti oltre.

File o cartella	A cosa serve	Righe
`www/index.html`	Strumento: orizzonte, mappa, sensori, filtri, avvisi, navigazione, circuito, diario, traffico, impostazioni, Info	5.192
`www/app.html`	Piano (iframe): rotte, campi, meteo, METAR/TAF, NOTAM, spazi aerei openAIP/OFM, briefing, Calcola rotta, profilo	2.750
`www/regole.js`	Regole per tipo di volo (VDS basico, avanzato, VFR; DPR 133/2010), giudizio sugli spazi aerei	92
`www/carburante.js`	Carburante e alternati in tre fasce (comodo, riserva, critico), campi con carburante	216
`www/prestazioni.js`	Salita e discesa per quota di densità e peso, giri di 360°, salita graduale	46
`www/licenza.js`	Codice del telefono, prova e abbonamento (spenti: `abbonamento:false`), «Offrimi un caffè»	205
`www/riservato.js`	Menu riservato: codice del telefono, richiesta, verifica del codice HZ1 (ECDSA), revoche	134
`www/guida.js`	Guida in linea, tour guidato, catalogo dei velivoli, «Prima del volo», check list a fasi	423
`www/impostazioni.js`	Impostazioni ordinate e pieghevoli, «Prima di partire», «In volo», tasti grandi	119
`www/schede.js`	Caselle dei dati a scelta (completa e semplice, al massimo 8)	99
`www/schermo.js`	Tema «pieno sole», tablet: schermo diviso regolabile, tocco per scambiare	120
`www/livelli.js`	Livelli della mappa e disegno degli spazi aerei e dei NOTAM sulla mappa	107
`www/emergenza.js`	Zona di planata vera, campi raggiungibili, modo EMERGENZA e MAYDAY	191
`www/vento.js`	Vento sulla pista (componenti), METAR sulla mappa, pagina del vento	142
`www/voce.js`	Voce su richiesta, check list a voce, METAR a voce, scadenze, rivedi un volo in 3D	92
`www/radio.js`	Chiamata radio nel circuito e pagina FREQUENZE	152
`www/frequenze.js`	Frequenze dei campi (generato da `tools/frequenze_ourairports.py`)	4
`www/stimati.js`	Riporto di posizione, ETA/ATA in UTC, autonomia	124
`www/note.js`	Punti VFR sulla mappa e NOTE ENTE (QNH, squawk, readback)	97
`www/squadriglia.js`	Aerei amici riconosciuti nel traffico	107
`www/carta.js`	Carta VFR open flightmaps: plugin nativo o worker web, ciclo AIRAC	71
`www/carta-web.js`	Worker della carta VFR: sqlite-wasm + OPFS	82
`www/orologio.js`	Orologio Garmin (attivo solo con il modulo nativo di HORIZON 2)	125
`www/spazi3d.js`	Vista 3D degli spazi aerei e voli rivisti (three.js)	338
`www/sw.js`	Service worker: cache per l'uso senza internet	33
`www/version.js`	Versione e canale (riscritto dai workflow)	1
`www/capacitor.js`	Runtime di Capacitor (generato, vendorizzato)	3 (9 KB)
`www/sqlite3.js`	sqlite-wasm 3.53.4 (vendorizzato, Apache-2.0)	17.393 (628 KB)
`www/three.min.js`	three.js r160 (vendorizzato, MIT)	7 (654 KB)
`android/app/src/main/java/it/amicoen/horizon`	10 classi Java: MainActivity e 9 file dei plugin (Gdl90Parser compreso)	1.334

© 2026 Enrico D'Amico · Tutti i diritti riservati

File o cartella	A cosa serve	Righe
`android/app/src/h2/java/it/amicoen/horizon/GarminPlugin.java`	Orologio Garmin (solo HORIZON 2, `-Ph2`)	147
`android/app/build.gradle`	Compilazione, firma, variante H2 (`applicationId` horizon2, SDK Connect IQ 2.4.0)	71
`garmin/source/HorizonApp.mc`	App per l'orologio: sensori, messaggi, tasti, scatola nera	289
`garmin/source/HorizonView.mc`	Schermata dell'orologio e gestione dei tasti	170
`garmin/manifest.xml`	Manifest Connect IQ (fenix 7/7S/7X, Pro, epix 2; permessi)	26
`.github/workflows`	8 workflow (vedi capitolo 10)	699
`tools/test_assetto.js`	Banco di prova dell'assetto sul codice vero di index.html	185
`tools/registrazioni`	2 registrazioni vere dei sensori (Samsung S23 FE)	1.890
`tools/emu_test.sh`	Collaudo dell'APK sull'emulatore (adb)	46
`tools/emu_cdp.py`	Collaudo nella WebView via DevTools (CDP)	210
`tools/licenze.py`	Chiavi ECDSA, genera e verifica i codici HZ1	102
`tools/licenze_azione.py`	Azioni a mano del workflow Licenze (genera, revoca, ripristina, manda)	91
`tools/posta_licenze.py`	Posta delle licenze: prezzi, codici, piste, riepilogo (IMAP/SMTP)	416
`tools/registro_unisci.py`	Unione di due copie del registro delle licenze	37
`tools/gmail-licenze/HORIZON-Licenze.gs`	Apps Script: sveglia ogni minuto, promemoria nel Calendar	138
`tools/piste_piloti.py`	Archivio comune delle piste rilevate e dei campi usati	162
`tools/campi_ourairports.py`	Campi da OurAirports (Italia, Francia, Alpi)	90
`tools/unisci_campi.py`	Unisce i campi nuovi a `FIELDS_RAW` delle due pagine	65
`tools/frequenze_ourairports.py`	Genera `www/frequenze.js`	52
`tools/ofm_openair.py`	Spazi aerei open flightmaps in OpenAir (riserva notturna)	77
`tools/confronta_spazi.py`	Confronto openAIP / open flightmaps	207
`tools/carburante_openaip.py`	Campi con carburante da openAIP → `dati/carburante.json`	32
`tools/prepare_www.py`	Prepara `www/` dai sorgenti esterni (ramo v2) con il runtime Capacitor	22
`tools/build_manuale_app.py`	Manuale dell'app in PDF	656
`tools/build_dossier_tecnico.py`	Questo documento	966
`.claude/skills/horizon/SKILL.md`	Mappa del progetto e regole di lavoro	78
`README.md`	Presentazione, canali, struttura, licenza	44
`PRIVACY.md`	Informativa sulla privacy	77

Altre cartelle: `docs/play/` (scheda e immagini del Play Store, note per i tester in `docs/play/novita/whatsnew-it-IT`), `docs/licenze/` (immagine dei passi di pagamento usata nella mail dei prezzi), `docs/garmin/` (immagini per lo store Connect IQ), `docs/dossier/` (questo documento, figure e sorgenti), `keystore/strumento.keystore.enc` (chiave di firma cifrata), `garmin/devkey.der.enc` (chiave da sviluppatore Garmin cifrata).

© 2026 Enrico D'Amico · Tutti i diritti riservati

2.1 Servizi esterni chiamati dall'app

Servizio	Indirizzo	A cosa serve	Quando
Open-Meteo	`api.open-meteo.com/v1/forecast`	Meteo del modello sui campi (vento, nubi) e controllo della rotta alle quote (1000–700 hPa)	Piano: al cambio dei campi (al più ogni 20 min) e a richiesta; cache «network first» nel service worker
aviationweather.gov	`/api/data/metar`, `/api/data/taf`	METAR e TAF dei campi vicini alla rotta (fino a 14 stazioni)	Piano, solo dentro lo strumento (`IA517.http`), al più ogni 10 min
openAIP	`storage.openaip.net/openaip-system-exports/xx_asp_v2.txt`, `api.core.openaip.net`	Spazi aerei Italia + fascia alpina (Francia, Svizzera, Austria, Slovenia); API con chiave del pilota come ripiego	A ogni ciclo AIRAC o a richiesta; sul web dalla copia notturna `dati/`
open flightmaps	`storage.googleapis.com/snapshots.openflightmaps.org/live/AIRAC/…`	Carta VFR MBTiles (≈ 105 MB) e, dalla copia notturna, spazi aerei di riserva	Scarico a mano della carta; spazi dal workflow notturno
AWS Terrain Tiles	`s3.amazonaws.com/elevation-tiles-prod/terrarium/z/x/y.png`	Rilievo (TAWS, orizzonte 3D, quote dei campi, rotte)	Continuamente attorno all'aereo; cache «cache first» (4000 tessere)
Tessere della mappa	`tile.opentopomap.org`, `tile.openstreetmap.org`, `ibasemaps-api.arcgis.com` (Esri, con chiave), `wms.pcn.minambiente.it` (IGM, ortofoto, solo app)	Mappa di base	Mentre la mappa è visibile; «Scarica per il volo» prepara le zone
OGN	`live.glidernet.org/lxml.php`	Traffico FLARM/OGN	Ogni 6 s in volo o con la pagina TRAFFICO aperta, se il GDL90 tace; zona arrotondata a 0,1°
adsb.lol	`api.adsb.lol/v2/point/lat/lon/20`	Traffico ADS-B/MLAT	Come OGN
Overpass (OSM)	`overpass-api.de/api/interpreter`	Nomi di paesi, laghi e cime per i punti disegnati	Quando si disegna una rotta (riquadro arrotondato a 0,1°)
Web3Forms	`api.web3forms.com/submit`	Segnalazioni, registrazioni dei sensori, piste e campi condivisi, richieste di licenza	Su azione del pilota (piste e campi solo con consenso, a terra)
FormSubmit	`formsubmit.co/ajax/…`	Riserva di Web3Forms (usata solo se la chiave Web3Forms è vuota)	Come sopra
GitHub API	`api.github.com/repos/amicoen-ops/HORIZON/releases/latest`	Controllo degli aggiornamenti (solo APK di GitHub)	All'avvio con la rete e da Info › Aggiornamenti
GitHub raw	`raw.githubusercontent.com/amicoen-ops/HORIZON/licenze/licenze/revocate.json`	Licenze revocate	15 s dopo l'avvio (al più una volta al giorno) e all'apertura del menu riservato
Sito HORIZON	`amicoen-ops.github.io/HORIZON/dati/carburante.json`, `version.js?chk=`	Campi con carburante; nuova versione web	Una volta al giorno; all'avvio (solo web)
Desk Aeronautico	`www.deskaeronautico.it`	Solo collegamenti (NOTAM, meteo, briefing): i NOTAM si incollano nel Piano	Su tocco del pilota
Google Fonts	`fonts.googleapis.com`	Caratteri Barlow Semi Condensed e Oxanium (Piano)	All'apertura, poi dalla cache

© 2026 Enrico D'Amico · Tutti i diritti riservati

3. Sensori e acquisizione dei dati

È la parte più delicata dello strumento: tutto quello che il pilota vede viene da qui. Ogni sensore arriva per una strada sua (API del browser, evento di un plugin nativo, messaggio dell'orologio), viene trasformato da una funzione precisa e finisce nello stato `S`, da cui leggono disegno, caselle e avvisi. La tabella riassume; le sezioni seguenti entrano nel dettaglio.

Sensore	Sorgente / evento	Frequenza	Funzione	Dove finisce
GPS del telefono	`navigator.geolocation.watchPosition` (alta precisione, `maximumAge 0`)	≈ 1 Hz (dal telefono)	`PH.onPos` → `PH.fixIn`	`S.lat/lon/gs/trk/alt/vs`, diario, vento
Garmin GLO 2	`GloSerialPlugin` evento «nmea» (web: Web Serial)	gruppi ogni 100 ms; fix 1–10 Hz	`PH.gloLine` → `PH.fixIn`	come sopra, `S.sats`, `S.hdop`
Orologio Garmin	`GarminPlugin` evento «dati»	≈ 1 Hz	`dati()` in `orologio.js` → `PH.fixIn`	come sopra; quota baro; bussola
Accelerometro e giroscopio	evento `devicemotion`	60–200 Hz	`PH.onMotion` (Mahony)	`S.roll/pitch/slip/hdg`, vario
Bussola	`CompassPlugin` «rot» (20 Hz) o `deviceorientation(absolute)`	≤ 20 Hz	`PH.magFromR`	`PH.mag.hdg` → `S.hdg` a terra
Barometro del telefono	`BarometroPlugin` evento «p» {hPa} (`TYPE_PRESSURE`)	≈ 5 Hz	`PH.baroIn` → `PhVario.correct`	`S.alt`, `S.vs`; scarto tenuto giusto dal GPS
Quota e vario	barometro del telefono se c'è, altrimenti GPS; accelerazione verticale; barometro dell'orologio (corretto)	fix + eventi di moto	`PhVario`	`S.alt`, `S.vs`
Traffico GDL90	`Gdl90Plugin` evento «traffico» (UDP 4000)	per pacchetto	`PH.gdlIn` → `PH.trfEval` (1 Hz)	`PH.TRF.tg`, triangolo, mappa
Stato del telefono	`StatoTelefonoPlugin.get()`	ogni 5 s	`PH.statoPoll`	riga di stato, qualità GPS

3.1 Posizione: GPS del telefono, Garmin GLO 2, orologio

Ogni fonte produce un oggetto «fix» con la stessa forma: `{src, lat, lon, msl, speed, heading, acc, t, recv}` (più `sats` e `hdop` per il GLO 2). `acc` è la precisione stimata in metri ed è il criterio di scelta. Tutte le fonti passano da `PH.fixIn(fx)`, che memorizza il fix in `PH.fixes[src]` e lo applica solo se `PH.srcPick()` dice che quella è la fonte in uso.

© 2026 Enrico D'Amico · Tutti i diritti riservati

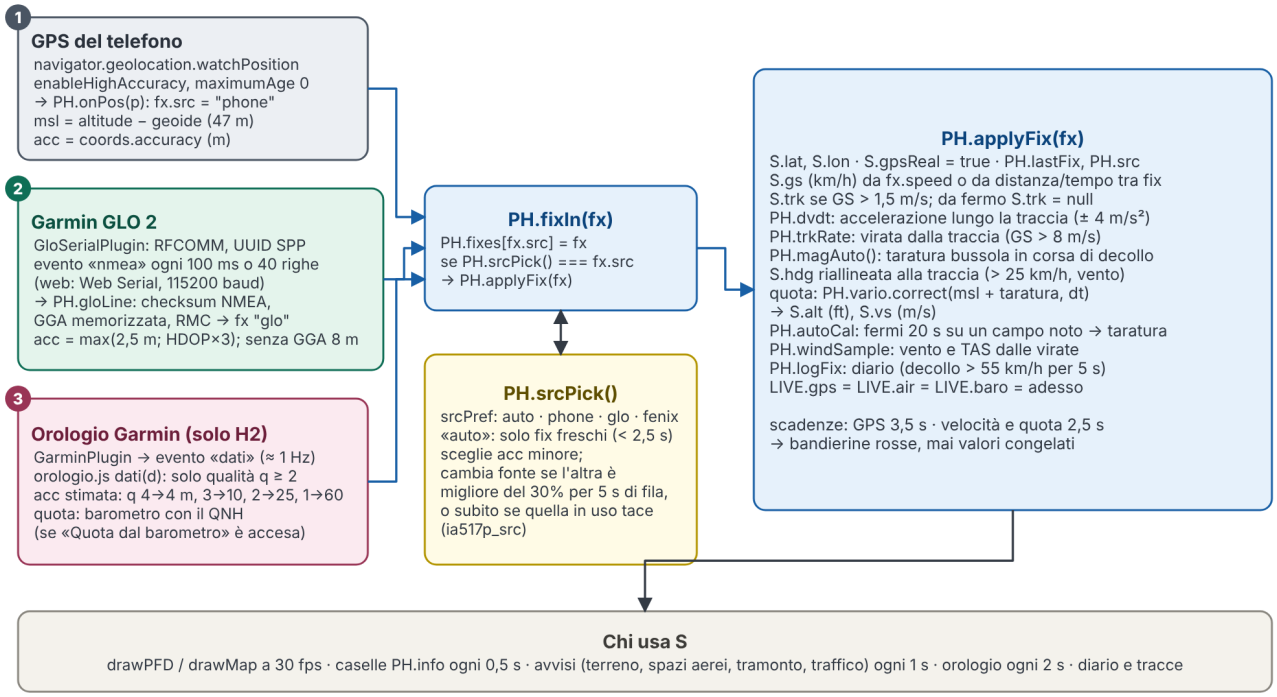


Figura 2 — Fonti di posizione e scelta della migliore: dal fix di ogni fonte a `PH.fixIn`, `PH.srcPick` e `PH.applyFix`, fino a chi legge lo stato `S`.

GPS del telefono

- Avvio in fondo alla parte telefono: `watchPosition(PH.onPos, PH.onPosErr, {enableHighAccuracy:true, maximumAge:0, timeout:15000})`.
- La quota del browser è sull'ellissoide WGS84: `PH.onPos` sottrae un geoide fisso `PH.geoid` (47 m di serie, `ia517p_geoid`), poi la taratura sul campo corregge il resto.
- Con il plugin `StatoTelefono` (solo APK) la riga di stato mostra quanti satelliti sono usati nel fix (da `GnssStatus`) e la colora: verde se ≥ 7 satelliti e precisione ≤ 15 m (`PH.GPS_Q`).

Garmin GLO 2

- App Android: `GloSerialPlugin.connect({address})` apre un socket RFCOMM verso l'UUID SPP `00001101-…`, prima cifrato e poi, se fallisce, «insecure». Un thread legge le righe ASCII che iniziano con `$` e le manda in gruppi (evento «nmea» `{lines:[…]}`) ogni 100 ms o ogni 40 righe. Se il collegamento cade arriva l'evento «state» `{connected:false, error}`. Il permesso `BLUETOOTH_CONNECT` è chiesto al primo uso.
- Browser: `navigator.serial.requestPort` con il filtro sulla classe SPP, `port.open({baudRate:115200})`, lettura in streaming con `TextDecoder`.
- `PH.gloLine` controlla il checksum NMEA, tiene l'ultima GGA (qualità, satelliti, HDOP, quota MSL) e, a ogni RMC valida (stato A), crea il fix `glo` con velocità in nodi × 0,514444 e `acc = max(2,5 m, HDOP × 3)` (8 m se manca una GGA fresca). Ogni 5 s, se il pilota lo vuole, si ritenta il collegamento.

Orologio Garmin (HORIZON 2)

- `orologio.js` si attiva solo se `Capacitor.isPluginAvailable('Garmin')`. Dei dati dell'orologio usa la posizione solo con qualità ≥ 2 e stima la precisione dalla qualità Garmin: 4 → 4 m, 3 → 10 m, 2 → 25 m, 1 → 60 m (`QM`).
- Se «Quota dal barometro» è accesa (di serie sì), la quota del fix è barometrica: `44330,77 · (1 − ((p + pOfs) / (QNH·100))^0,190263)` con la pressione `p` in Pa e il QNH dello strumento. `pOfs` è lo scarto del barometro dell'orologio imparato dal barometro del telefono quando sono freschi tutti e due (media sulle prime 30 letture, poi circa 5 minuti; salvato in `ia517p_fenix_pofs`). Misurato: fenix 7 −4,4 hPa (circa 37 m); la calibrazione dell'orologio non lo toglie.
- Il capitolo 8 descrive il protocollo completo.

Scelta della fonte: PH.srcPick

- `PH.srcPref` (`ia517p_src`): «auto» oppure una fonte fissa (`phone`, `glo`, `fenix`); con una fonte fissa che tace compaiono le bandierine rosse.

© 2026 Enrico D'Amico · Tutti i diritti riservati

- «auto»: tra i fix freschi (ricevuti da meno di 2,5 s) si prende quello con `acc` minore. Per non saltare avanti e indietro si cambia solo se il candidato resta migliore del 30% per 5 s di fila; se la fonte in uso tace si passa subito alla migliore.
- `PH.applyFix` aggiorna posizione, velocità (da `speed` o, se manca, da distanza e tempo tra due fix), traccia (solo sopra 1,5 m/s), accelerazione lungo la traccia `PH.dvdt`, velocità di virata dalla traccia `PH.trkRate`, prua riallineata alla traccia (sopra 25 km/h, corretta con il vento stimato), quota e vario, taratura, vento, diario. Quando cambia la fonte della quota il filtro del vario riparte da capo.

Taratura della quota (`PH.autoCal`). Fermi (meno di 0,8 m/s, precisione entro 20 m) per almeno 20 s e 10 fix, entro 700 m da un campo con quota nota: la differenza tra quota del campo e media GPS (se entro 120 m) diventa l'offset di quella fonte (`PH.altOfsBy[src]`, `ia517p_altofs2`) e compare «QUOTA TARATA SU …».

Vento e velocità vera senza Pitot (`PH.windSample`). Un campione di velocità al suolo (Est, Nord) al secondo, fino a 3 minuti e 100 campioni. Con almeno 20 campioni e una virata vera (le direzioni non tutte allineate), si risolve ai minimi quadrati il cerchio |v − vento| = TAS: accettato se TAS tra 15 e 70 m/s e vento sotto 25 m/s, poi filtrato (80/20). Prima del vento stimato vale quello previsto da Open-Meteo per la destinazione (`PH.applyForecast`).

Diario (`PH.logFix`). Il volo inizia dopo 5 s sopra 55 km/h (decollo, con il campo più vicino entro 3 km); il volo in corso è salvato in `ia517p_curflight` e ripreso se l'app si riavvia entro 15 minuti. Le tracce degli ultimi 50 voli (`PH.TRK_KEEP`) stanno in IndexedDB ed esportano in GPX e KML.

3.2 Assetto: accelerometro e giroscopio

L'assetto viene da `PH.onMotion`, chiamata a ogni evento `devicemotion`. Usa lo stesso filtro della scheda sensori dello strumento fisico (Mahony a quaternioni, `ahrs_core.h`) con la compensazione delle accelerazioni in virata fatta con la velocità GPS al posto del Pitot. Convenzioni: assi dell'aereo avanti, destra, giù; forza specifica a riposo e livellato (0, 0, −1 g).

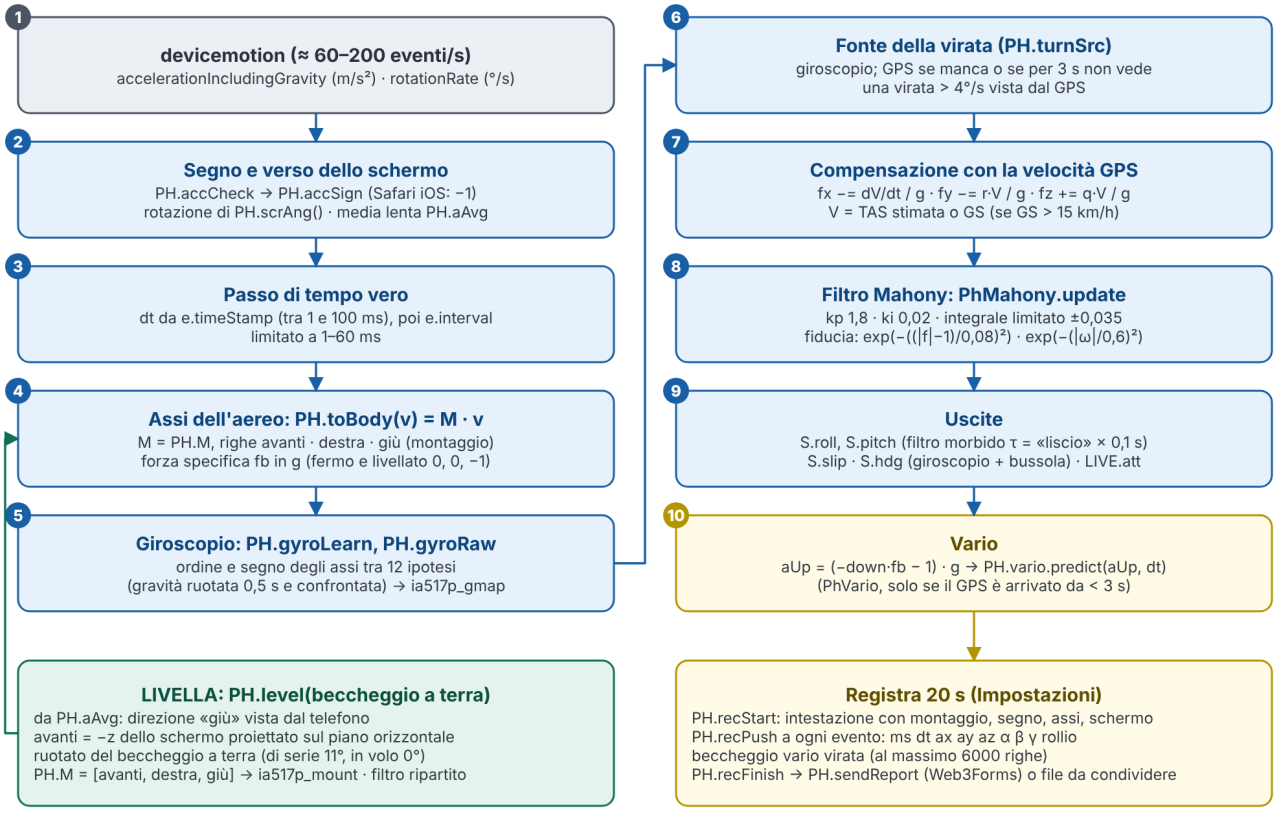


Figura 3 — Dai sensori di movimento all'assetto: i dieci passi di `PH.onMotion`, il livellamento (LIVELLA) e la registrazione di 20 s.

- **Segno dell'accelerometro.** Safari su iPhone dà `accelerationIncludingGravity` col segno opposto ad Android: si parte da `PH.accSign = −1` su iOS e lo si conferma da soli col telefono dritto in verticale (`PH.accCheck`, 40 voti contrari per invertirlo).
- **Verso dello schermo.** I vettori arrivano negli assi del telefono in verticale; `PH.scrAng()` (0, 90, 180, 270°; su iOS da `window.orientation`) li ruota negli assi dello schermo. Una rotazione dello schermo fa ripartire il

© 2026 Enrico D'Amico · Tutti i diritti riservati

filtro (`PH.onRotate`).

- **Passo di tempo vero.** Molti Android dichiarano `interval` 16 ms ma mandano 100–200 eventi al secondo: `dt` si misura da `e.timeStamp` (valido tra 1 e 100 ms), altrimenti si usa `interval`, sempre limitato a 1–60 ms. Con il valore dichiarato il giroscopio veniva contato 2–3 volte.
- **Montaggio (`PH.M`).** Matrice le cui righe sono gli assi avanti, destra, giù dell'aereo negli assi dello schermo; di partenza il telefono in verticale con lo schermo verso il pilota (`[[0,0,−1],[1,0,0],[0,−1,0]]`). `PH.toBody(v)` = M·v porta accelerazione e rotazione negli assi dell'aereo.
- **Ordine degli assi del giroscopio (`PH.gyroLearn`).** Lo standard W3C è x = beta, y = gamma, z = alpha, ma alcuni WebView (Samsung S23 FE) li danno come alpha, beta, gamma = x, y, z. Per 6 ordini × 2 segni la gravità di inizio finestra viene ruotata con il giroscopio per 0,5 s (dG/dt = −ω × G) e confrontata con quella misurata: l'ipotesi giusta resta dove la vede l'accelerometro. Servono almeno 9° di rotazione per finestra e 4 finestre; il segno si cambia solo con una prova netta. Il risultato è in `ia517p_gmap`.
- **Virata dal GPS.** Se il giroscopio manca, o per 3 s non vede una virata che il GPS vede chiaramente (oltre 4°/s, meno del 30% della rotazione attesa), la velocità di virata della traccia (rotazione attorno alla verticale) si porta negli assi dell'aereo con l'assetto stimato in quel momento: `[−ω sinθ, ω sinφ cosθ, ω cosφ cosθ]`. Usare l'inclinazione teorica della virata coordinata faceva beccheggiare l'orizzonte di circa 28° in macchina.
- **Compensazione.** Con V = TAS stimata o GS (sopra 15 km/h): `fx −= dV/dt / g`, `fy −= r·V / g`, `fz += q·V / g`. Con la virata dal GPS, se il modulo della forza misurata è quello atteso √(1 + (V·ω/g)²), se ne tiene solo la direzione così l'inclinazione converge.
- **Filtro Mahony.** `kp` 1,8, `ki` 0,02 con integrale limitato a ±0,035 rad/s; la fiducia nell'accelerometro scende con |f| lontano da 1 g (scala 0,08 g) e con la rotazione (scala 0,6 rad/s): `trust = exp(−e1²)·exp(−e2²)`. Senza accelerometro credibile (|f| fuori da 0,3–3 g) integra solo il giroscopio più il bias stimato.
- **Uscite.** Rollio e beccheggio passano da un filtro del primo ordine regolabile («orizzonte più morbido», τ = `PH.smooth` × 0,1 s, di serie 0,3 s) e vanno in `S.roll`, `S.pitch`; lo sbandamento `S.slip` dalla forza laterale; la prua integra il giroscopio tra un fix e l'altro e, a terra o lenti, viene tirata verso la bussola. `LIVE.att` scade in 0,7 s.

LIVELLA

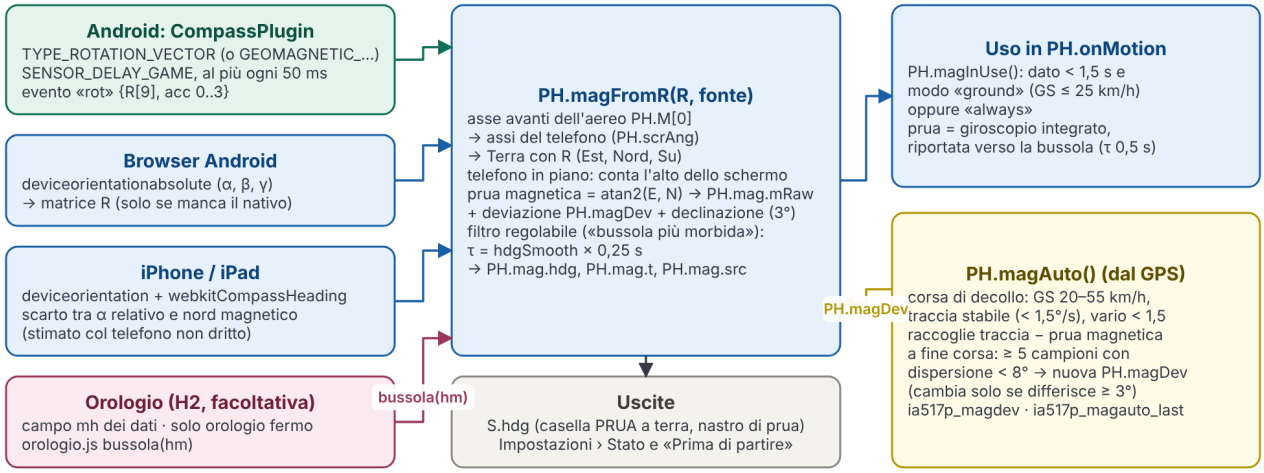
Il tasto LIVELLA (giallo e nero, chiede conferma; non disponibile in demo e simulazione) chiama `PH.level(gp)`: dalla media lenta dell'accelerometro `PH.aAvg` ricava la direzione «giù» vista dal telefono, prende come «avanti» l'asse −z dello schermo proiettato sul piano orizzontale (se il telefono è quasi in piano, l'alto dello schermo) e lo ruota del beccheggio a terra (`gp`: di serie 11° per un biciclo a terra, 0° se si livella in volo). Ne esce la nuova `PH.M`, salvata in `ia517p_mount`; il filtro riparte già allineato (`levelFrom`). Se l'accelerazione media non è tra 7 e 12,5 m/s² compare «TIENI FERMO IL TELEFONO E RIPROVA».

Registrazione dei sensori («Registra 20 s e invia»)

Da Impostazioni il pilota registra 20 s di dati grezzi: `PH.recStart` scrive l'intestazione (montaggio, beccheggio a terra, segno, ordine degli assi, verso dello schermo, liscio, stato del GPS, user agent); `PH.recPush`, chiamata da `onMotion`, aggiunge per ogni evento una riga `ms dt ax ay az alpha beta gamma rollio beccheggio vario virata` (al massimo 6000 righe). La fine non dipende dagli eventi: un timer chiama `PH.recFinish`, che manda tutto con `PH.sendReport` (Web3Forms, entro 20 s) oppure apre la condivisione del file. Le registrazioni ricevute vanno in `tools/registrazioni/` e il banco di prova le rigioca nel codice vero (capitolo 11).

© 2026 Enrico D'Amico · Tutti i diritti riservati

3.3 Bussola



Il telefono non ha un modulo «barometro» nell'app: la quota viene dal GPS (o dal barometro dell'orologio in HORIZON 2).

La bussola serve soprattutto a terra e lenti: in volo la prua viene dal giroscopio riallineato alla traccia GPS corretta col vento stimato.

Figura 4 — Bussola: le quattro sorgenti (plugin nativo, browser Android, iPhone, orologio) arrivano a `PH.magFromR`; uso nella prua e taratura automatica in corsa di decollo.

- **CompassPlugin**: sensore `TYPE_ROTATION_VECTOR` (o, se manca, `GEOMAGNETIC_ROTATION_VECTOR`) a `SENSOR_DELAY_GAME`, eventi «rot» al più ogni 50 ms con la matrice R (telefono → Est, Nord magnetico, Su) e la precisione 0–3. Si sospende in pausa e riprende con l'app.
- Nel browser: `deviceorientationabsolute` (o `deviceorientation` con `absolute`), ignorato se c'è il modulo nativo. Su iPhone `alpha` è relativo: lo scarto verso `webkitCompassHeading` si stima quando il lato alto del telefono non è verticale e poi resta valido anche col telefono dritto.
- `PH.magFromR` porta l'asse avanti dell'aereo nella terna terrestre, calcola la prua magnetica, aggiunge deviazione (`PH.magDev`) e declinazione (`PH.decl`, 3° di serie) e la filtra (τ = `hdgSmooth` × 0,25 s). La bussola tiene la prua solo se `PH.magInUse()`: dato recente e modo «ground» (fino a 25 km/h) o «always».
- `PH.magAuto` sfrutta la corsa di decollo (20–55 km/h, traccia stabile): con almeno 5 campioni coerenti (dispersione sotto 8°) aggiorna la deviazione, senza domande.

© 2026 Enrico D'Amico · Tutti i diritti riservati

3.4 Quota, vario e QNH

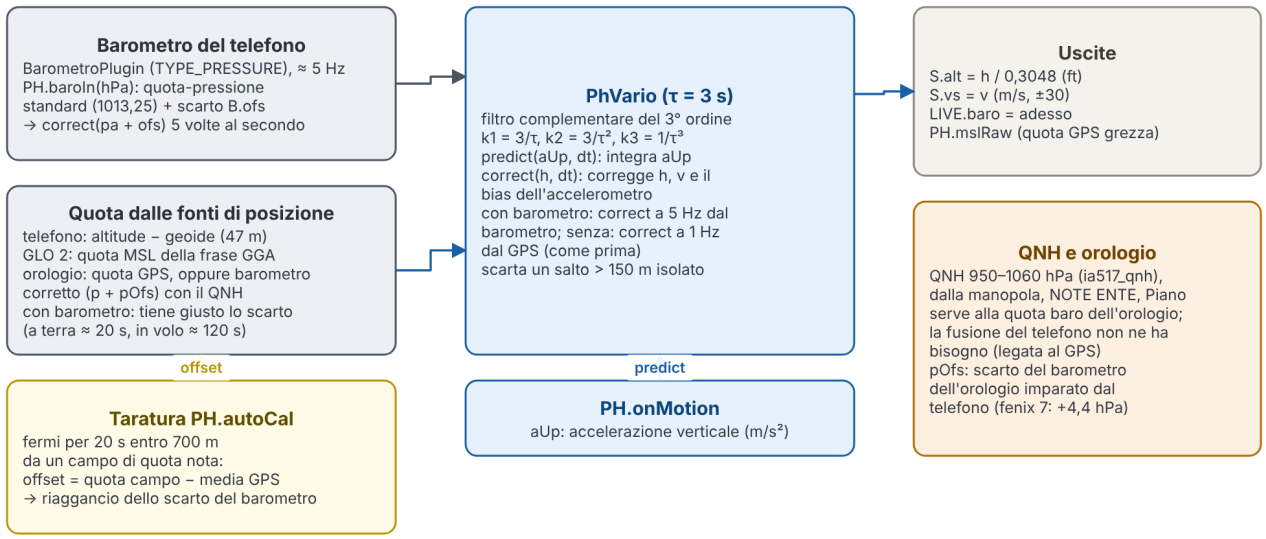


Figura 5 — Quota e vario: il barometro del telefono (5 Hz) corregge il filtro `PhVario`; il GPS tarato tiene giusto lo scarto. Senza barometro il filtro usa la quota GPS come prima.

`PhVario` è un filtro complementare del 3° ordine (τ = 3 s: guadagni 3/τ, 3/τ², 1/τ³) che stima quota, velocità verticale e bias dell'accelerometro. `predict` integra l'accelerazione verticale ricavata da `onMotion` solo se un fix è arrivato da meno di 3 s (altrimenti il vario si smorza verso zero); `correct` usa la quota della fonte in uso, scarta un salto isolato di oltre 150 m e riparte dopo tre salti o dopo un buco di oltre 3 s. Uscite: `S.alt` in piedi, `S.vs` in m/s (limitato a ±30).

Barometro del telefono (dal 9 novembre 2026). `BarometroPlugin.java` legge `TYPE_PRESSURE` circa 5 volte al secondo (evento «p» {hPa}) e si ferma quando l'app va in pausa. `PH.baroIn` trasforma la pressione in quota-pressione standard `44330 · (1 − (p/1013,25)^0,190263)` e, una volta agganciato lo scarto `PH.baro.ofs`, chiama `PhVario.correct(pa + ofs)` a 5 Hz. In `PH.applyFix`, con il barometro fresco, il GPS non corregge più il filtro ma solo lo scarto: `ofs += (quota GPS tarata − (pa + ofs)) · dt / τ`, con τ = 20 s a terra e 120 s in volo, ignorando i fix con precisione peggiore di 30 m. Lo scarto si riaggancia subito quando il filtro riparte (taratura sul campo, «Tara qui», cambio di fonte, avvio). Barometro fermo da 2 s, spento (`ia517p_baro`) o volo dimostrativo/simulazione: si torna al solo GPS. Così il vario viene dal barometro e la quota assoluta resta quella del GPS tarato, senza bisogno del QNH. Simulazione con GPS ±15 m: rumore del vario da ±2,4–3,6 a ±0,05–0,11 m/s, errore di quota da 5–8 a 2–5 m.

Il QNH (950–1060 hPa, di serie 1020, `ia517_qnh`) si imposta dalla manopola, dalle NOTE ENTE o dal Piano («Invia QNH» sui METAR) e serve alla quota barometrica dell'orologio e alla scritta sull'orizzonte; il codice di `applyNMEA` che converte la pressione con il QNH appartiene allo strumento fisico (scheda sensori via Raspberry). In Stato la riga «Barometro» dice se c'è, quanti hPa legge e se è in uso.

Misure vere (`docs/sensori/scarti.json`, 9 novembre 2026): Galaxy S23 FE 982,5 hPa → QNH 1021,5 contro il METAR di LIMF 1022 (−0,5 hPa); Galaxy S23 −0,4 hPa rispetto all'S23 FE; fenix 7 −4,4 hPa (circa 37 m), corretto da `orologio.js`. Ogni nuova prova su un dispositivo si aggiunge a quel file.

© 2026 Enrico D'Amico · Tutti i diritti riservati

3.5 Traffico GDL90 (SafeSky, SkyEcho, PilotAware)

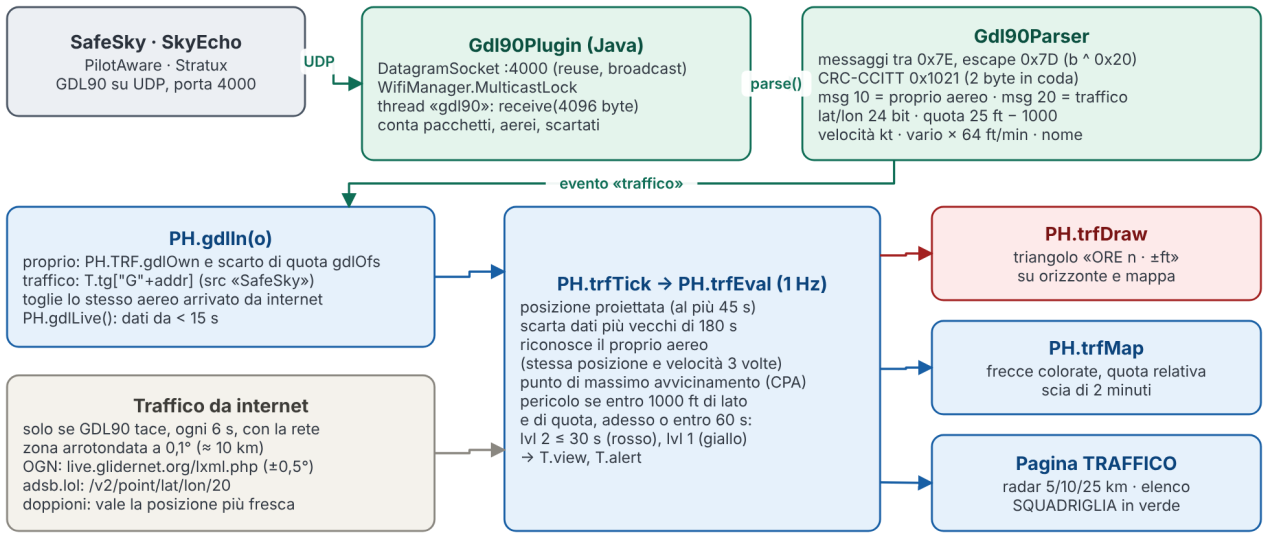


Figura 6 — Traffico: GDL90 via UDP 4000 (plugin e parser Java) oppure OGN e adsb.lol da internet, valutazione del pericolo e visualizzazione.

- `Gdl90Plugin.start({porta:4000})` apre un `DatagramSocket` riusabile con broadcast e prende un `MulticastLock` del Wi-Fi (senza, i pacchetti broadcast di SkyEcho e Stratux non arrivano).
- `Gdl90Parser.parse` separa i messaggi tra byte 0x7E, toglie l'escape 0x7D, verifica il CRC-CCITT (polinomio 0x1021, due byte in coda) e legge i messaggi 10 (proprio aereo) e 20 (traffico): indirizzo a 24 bit, lat/lon a 24 bit con segno (× 180 / 2^{23}), quota a passi di 25 ft − 1000, velocità in nodi, vario × 64 ft/min, traccia × 360/256, categoria, nominativo.
- `PH.gdlIn` tiene il proprio aereo (e lo scarto tra la sua quota e quella GPS, `gdlOfs`, per correggere le quote del traffico SafeSky) e mette ogni traffico in `PH.TRF.tg` togliendo lo stesso aereo arrivato da internet. Finché il GDL90 parla (ultimi 15 s) OGN e adsb.lol non si interrogano.
- `PH.trfEval` (1 Hz) proietta ogni traffico in avanti con la sua velocità (al più 45 s), scarta i dati più vecchi di 180 s, riconosce il proprio aereo (stessa posizione e velocità per 3 controlli), calcola il punto di massimo avvicinamento e dà il pericolo: entro 1000 ft di lato e di quota adesso o entro 60 s (rosso se entro 30 s). Il triangolo «ORE n · ±ft» compare sull'orizzonte e sulla mappa; la pagina TRAFFICO ha radar 5/10/25 km ed elenco.

3.6 Stato del telefono

`StatoTelefonoPlugin.get()` (ogni 5 s) restituisce batteria (percentuale, in carica, temperatura), rete (Wi-Fi/cellulare, internet validato), generazione e segnale del cellulare (5G da SS-RSRP, 4G da RSRP tra −120 e −80 dBm, 3G/2G da dBm) e, dal `GnssStatus.Callback`, satelliti usati e visti con il C/N0 medio (solo se aggiornati negli ultimi 10 s). Nessun permesso aggiuntivo oltre alla posizione. Nel browser `PH.statoPoll` usa `navigator.getBattery()` e `navigator.connection` quando ci sono. La riga di stato mostra «WIFI», «4G 63%», «BATT 18%» con i colori (rosso sotto 20%, giallo sotto 40%, rosso oltre 45 °C).

© 2026 Enrico D'Amico · Tutti i diritti riservati

4. Strumento e schermate

Le schermate di questo capitolo vengono dalla pagina vera servita in locale (Chromium, 412 × 900 punti, volo dimostrativo). In questo ambiente le tessere della mappa e del terreno non arrivano (rete chiusa): la mappa è senza fondo e l'orizzonte usa le colline sintetiche che il volo dimostrativo disegna proprio quando manca il rilievo.



Figura 7 — Orizzonte (PFD) nel volo dimostrativo con direct to LILE: sentiero magenta, CDI, destinazione



Figura 8 — Mappa mobile senza tessere (rete chiusa in questo ambiente)



Figura 9 — Interfaccia semplice

4.1 Orizzonte (PFD)

- `drawPFD` disegna nel tondo: cielo e terreno 3D (`drawTerrain` proietta il rilievo con `project`/`rel` e lo colora per quota; in volo giallo sotto 180 m e rosso sotto 60 m di margine, `TAWS_RED`/`TAWS_YEL`), ostacoli e cavi (solo nel demo), piste 3D con barra di arresto prevista, sentiero a riquadri magenta lungo la rotta (`drawPathway`), scala di beccheggio, arco di rollio, nastri di velocità (archi colorati del velivolo attivo) e quota, vario, nastro di prua, CDI con modi ENR/TERM/APR, indicatore di sentiero (VDI), barre del direttore di volo, marcatore della traiettoria, riquadri di destinazione, vento e fase del circuito.
- Senza Pitot il nastro di velocità mostra la velocità al suolo (`S.ias = S.gs`); la TAS stimata dalle virate compare nella casella GS.
- Interfaccia semplice (`PH.ui === 'simple'`, classe `ph-simple`): solo orizzonte, `drawPFD` viene chiamato con rotta e destinazione nascoste; avvisi e SOS restano.

4.2 Mappa

- `drawMap` a tutto riquadro (`mapClip`), traccia in alto o nord in alto con la mappa spostata a dito (`PH.pan`); 19 passi di zoom da 1 a 700 km.
- Basi (`PH.BASES`): OpenTopoMap, OpenStreetMap, Satellite Esri (con chiave del pilota), IGM 1:25.000 e ortofoto AGEA (WMS http, solo app), personalizzata (XYZ o WMS), solo rilievo, e la carta VFR open flightmaps locale (`ofm`, capitolo 6). Le tessere coprono 1,8 volte la scala per riempire gli angoli ruotati.
- Sopra la base: rilievo colorato, spazi aerei e NOTAM (`livelli.js`, attenuati oltre ±1000 ft dalla quota), campi e piste, punti VFR (`note.js`), zona di planata vera (`emergenza.js`), rotta e tappe, traffico con scia,

© 2026 Enrico D'Amico · Tutti i diritti riservati

METAR come anelli colorati per categoria VFR (`vento.js`). I livelli si spengono dal tasto ◧ senza spegnere gli avvisi.

4.3 Terreno (DEM) e TAWS

- Rilievo da AWS Terrain Tiles in formato Terrarium: `phDemFetch` scarica la tessera PNG, la decodifica (quota = R·256 + G + B/256 − 32768) in un `Int16Array` 256 × 256; al massimo 6 richieste insieme, 220 tessere in memoria (circa 28 MB), ritentativo dopo 60 s. `phDemAt` interpola in modo bilineare e scende ai livelli più grossolani se manca il dettaglio; restituisce almeno 0 m (il fondo del mare non conta) salvo con `raw`, che serve a riconoscere il mare.
- `PH.checkTerrain` (1 s, solo in volo con GPS vero): per t da 0 a 60 s a passi di 5 s proietta la posizione lungo la traccia con il vario; «TERRENO! CABRA» (avviso rosso, ripetuto ogni 5 s) se entro 30 s il margine scende sotto 100 ft, «ATTENZIONE TERRENO» sotto 300 ft. Tace vicino alla destinazione in circuito (`tawsInhibit`).
- Terreno davanti alla quota obiettivo, quota sicura delle tratte e Calcola rotta usano lo stesso `phDemAt`.

4.4 Avvisi

`PH.alert(chiave, livello, testo, ogni)` è l'unico punto di ingresso: a schermo, i gialli informativi (spazi aerei, tramonto, limite VDS) diventano bandierine dal bordo destro, aperte 3 s e poi linguette finché la condizione continua (spariscono dopo 5 s di silenzio); i rossi e i gialli di terreno restano al centro. La voce (`PH.say`, TextToSpeech nativo o `speechSynthesis`) e la vibrazione partono al più una volta per intervallo. Un tocco toglie l'avviso: per uno spazio aereo finché non se ne esce da 2 minuti, per gli altri 1 minuto. In HORIZON 2 l'avviso va anche all'orologio.

Avviso	Condizione	Livello
TERRENO! CABRA / ATTENZIONE TERRENO	margine sotto 100 ft entro 30 s / sotto 300 ft entro 60 s	W / C
DENTRO … / TRA 30 SECONDI …	spazio aereo con il giudizio di `IA517_RULES` per il tipo di volo, adesso, a metà anticipo e all'anticipo (di serie 60 s), margine di quota 100 ft	W se vietato, altrimenti C
SOPRA IL LIMITE VDS	VDS basico: quota sopra ostacolo più alto entro 5 km + limite del giorno (`vdsMaxAgl`)	C
TRAMONTO TRA 30 / 10 MINUTI	alba e tramonto NOAA semplificato (`phSun`)	C / W
Traffico	triangolo «ORE n · ±ft» (non passa da `PH.alert`)	giallo / rosso

4.5 Vista 3D, schede, schermo diviso, voce, emergenza

- **Vista 3D (`spazi3d.js`, caricato dal Piano).** `SP3D_open` costruisce con three.js r160 (caricato solo all'apertura) il terreno attorno alla rotta, gli spazi aerei come solidi tra base e tetto (tagliati a 10.000 ft di partenza, FIR/UIR esclusi), la rotta alla quota del piano, il velo del limite VDS e l'aereo; tratti di circa 100 km. `SP3D_replay` rivede un volo del diario. Non si apre in volo vero.
- **Caselle (`schede.js`).** Fino a 8 caselle a scelta, elenchi separati per l'interfaccia completa e quella semplice (`ia517p_cards`, `ia517p_cards_s`); toccare VENTO, DESTINAZIONE o AUTONOMIA apre la pagina relativa.
- **Tablet (`schermo.js`, `PH.layout`).** In orizzontale e oltre 820 punti lo schermo si divide: orizzonte e mappa (o stimati) affiancati; tocco breve sulla finestra grande per scambiarle, riga centrale da trascinare (`ia517p_split_r`).
- **Voce (`voce.js`).** `PH.sayNow` parla anche con la voce spenta su richiesta; check list guidata a voce; METAR letto a parole; scadenze personali (medico, attestato, assicurazione) avvisate a terra entro 30 giorni.
- **Emergenza (`emergenza.js`).** `PH.glideZone`: in 36 direzioni la discesa con efficienza × 0,75, corretta dal vento, fino a 200 ft dal terreno vero; `PH.glideFields`: campi con terreno libero lungo la strada e almeno 500 ft all'arrivo; `PH.emergStart` (tasto SOS da tenere premuto 0,8 s) apre la barra di emergenza con il MAYDAY compilato.
- **Simulazione.** `PH.setDemo(true)` è il volo dimostrativo (sensori finti, colline sintetiche senza rilievo, ostacoli e traffico finti); `PH.simStart` fa volare lo strumento lungo una rotta o un direct to con velocità, quota, vento e accelerazione (×2 di serie) scelti, con avvisi veri ma simulati; sotto il terreno la simulazione si ferma come impatto.

© 2026 Enrico D'Amico · Tutti i diritti riservati



Figura 10 — Impostazioni ordinate (sezioni pieghevoli)



Figura 11 — Guida in linea

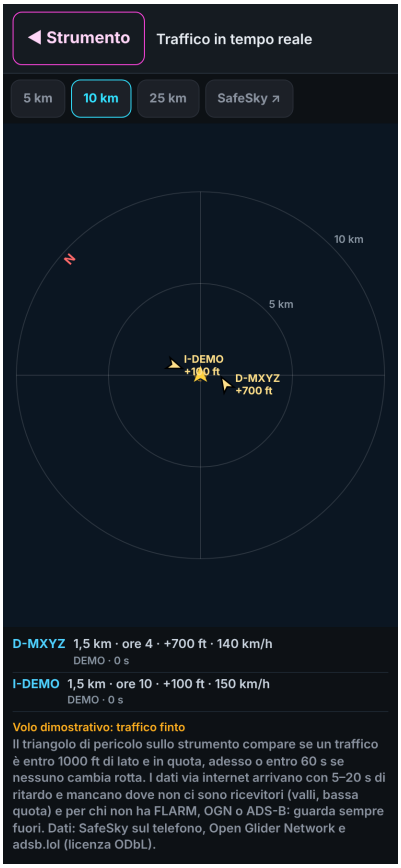


Figura 12 — Pagina TRAFFICO nel volo dimostrativo



Figura 13 — Tablet in orizzontale (1180 × 820): orizzonte e mappa affiancati con lo schermo diviso; mappa senza tessere per la rete chiusa.

© 2026 Enrico D'Amico · Tutti i diritti riservati

5. Piano di volo (app.html)

Il Piano è una pagina a parte caricata nell'iframe `#phAppFrame` dello strumento (tasto PIANO). Nasce come app web che parlava con il servizio del Raspberry: in versione telefono `api()` passa da `parent.IA517.api`, cioè `PH.handle`, che valida campi (`PH.valField`) e rotte (`PH.valRoute`) e li salva in `localStorage`. Il Piano legge dallo strumento il rilievo (`parent.phDemAt`), il velivolo attivo (`parent.IA517_AC`), il vento sulla pista (`parent.PH.ventoSvg`); lo strumento legge dal Piano gli spazi aerei veri (`IA517_asp`), i NOTAM con la riga Q validi oggi (`IA517_notam`), il calcolo della rotta per il direct to (`IA517_dctRoute`). Cinque schede: Piano, Briefing, Campi, Meteo, Strumento.

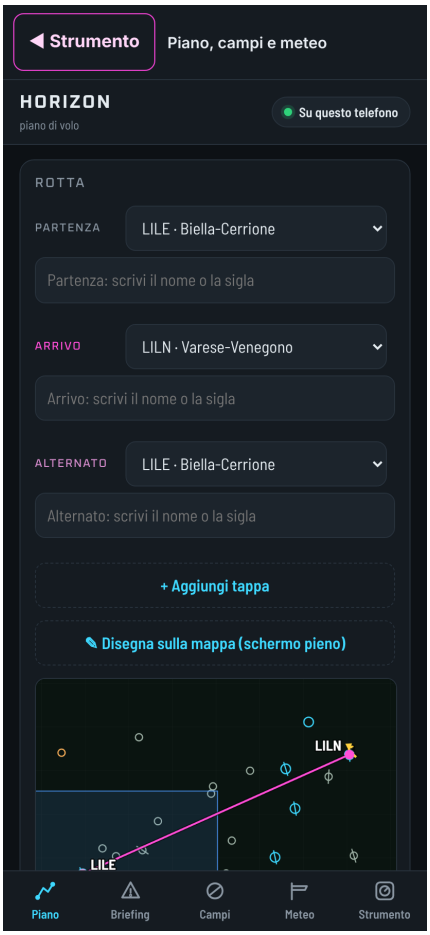


Figura 14 — Piano: partenza, arrivo, alternato, tappe e piccola mappa

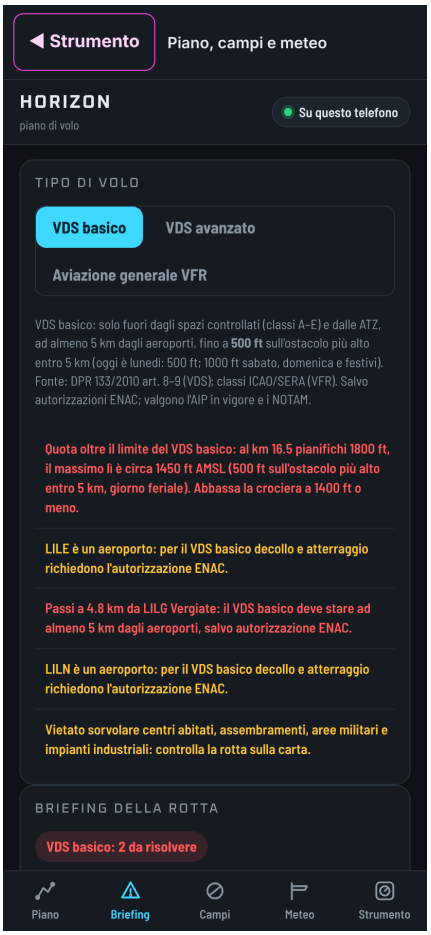


Figura 15 — Briefing (spazi aerei non scaricati in questo ambiente)

5.1 Rotta

- Partenza, tappe, arrivo e alternato si scelgono scrivendo nome o sigla tra circa 1950 campi (`FIELDS_RAW`, uguale nelle due pagine) più quelli del pilota.
- «Disegna sulla mappa» apre la rotta a schermo pieno (`rdOpen`): un tocco aggiunge un punto, trascinare la linea inserisce una tappa, i punti liberi prendono il nome di paesi, laghi e cime da Overpass. Per ogni tratta `rdLegAnalyze` calcola prua (con il vento alle quote), minuti, distanza, quota sicura sul terreno e spazi attraversati; `rdVert` verifica le salite e discese possibili col velivolo e mette giri di 360° dove servono.
- «Calcola rotta» (`rdAround`, ricerca A* su griglia con heap): aggira P, R e gli spazi vietati per il tipo di volo, passa dalle valli rispettando il margine sul terreno, evita il mare al largo (e lo permette caro, ×9, solo se non c'è altra strada), costa di più lontano da campi o terreno pianeggiante. Per il VDS basico una tratta si spezza solo se supera il limite di oltre 500 ft (`RD_VDS_TOL`).
- Il direct to dello strumento (`PH.dctSmart`) parte subito in linea retta e intanto chiede al Piano la stessa rotta calcolata e la quota migliore.

© 2026 Enrico D'Amico · Tutti i diritti riservati

5.2 Briefing, meteo, spazi aerei, NOTAM

- **Briefing** (`runBriefing`): profilo verticale con terreno, spazi aerei lungo la rotta campionata ogni 250 m, quota di crociera e giudizio per tipo di volo (`IA517_RULES`), più la Vista 3D.
- **Meteo**: Open-Meteo sui campi del piano (fino a 24), METAR e TAF veri da aviationweather.gov (solo dentro lo strumento), categoria VFR/MVFR/IFR/LIFR (`vfrCat`), TAF interpretati (FM, BECMG, TEMPO/PROB come peggio possibile, `tafAt`) e TAF entro 40 km all'ora di passaggio (`wxTafRoute`); METAR a parole e a voce.
- **Meteo della rotta alle quote** (`rdWxCheck`): Open-Meteo con vento, temperatura e geopotenziale da 1000 a 700 hPa, nubi basse e medie, visibilità, CAPE, zero termico, all'ora in cui si passa su ogni tratta.
- **Spazi aerei**: file OpenAir dell'Italia di openAIP (nell'app direttamente, sul web dalla copia notturna `dati/` perché openAIP non permette la lettura dal browser), più la fascia alpina dei Paesi vicini; ripiego sull'API openAIP con la chiave del pilota, su un file o su un indirizzo. Riserva open flightmaps unita sul telefono (`aspMerge`: base più bassa, tetto più alto, classe più restrittiva).
- **NOTAM**: si incollano (copiati dal briefing ufficiale o da Desk Aeronautico); `parseNotams` legge riga Q (posizione, raggio, quote), A), B), C), E) e li confronta con la rotta.

5.3 Velivolo, carburante, prestazioni, diario

- Profili dei velivoli (`PH.AC_SCHEMA`): identità e categoria (decide le regole), check list a fasi, velocità VS0…VNE (archi colorati), prestazioni, serbatoi e consumi, pesi e bracci, transponder e radio.
- `carburante.js`: carburante all'arrivo in tre fasce (comodo, riserva, critico sotto 10 minuti), alternati con carburante lungo la rotta da `dati/carburante.json` (openAIP).
- `prestazioni.js`: rateo di salita che cala con la quota di densità e cambia con il peso, discesa massima, giri di 360° da 10° a 20° d'inclinazione, salita graduale.
- Carico e centraggio (`renderWB`) con i limiti del baricentro del profilo. Il diario è nello strumento: voli automatici, tracce GPX/KML, esportazione CSV.

© 2026 Enrico D'Amico · Tutti i diritti riservati

6. Volare senza internet: cache, carta VFR, spazi aerei

6.1 Service worker

Richiesta	Strategia	Cache
Pagine e moduli del sito (stessa origine)	«stale while revalidate»: dalla cache subito, aggiornata in sottofondo	`ia517-app-1.0.0` (precaricata con `CORE` all'installazione, solo web)
Terreno (`s3.amazonaws.com/elevation-tiles-prod`)	«cache first»	`ia517-dem`, al massimo 4000 tessere
OpenTopoMap, OpenStreetMap	«cache first»	`ia517-map`, al massimo 3000 tessere
Open-Meteo	«network first», cache se offline	`ia517-wx`
Google Fonts	«cache first»	`ia517-fonts`, 40 voci
URL con `?chk=`	sempre dalla rete (controllo della versione)	—

Nell'app (host `localhost` senza porta) le pagine non si mettono in cache perché sono già nell'APK; terreno, mappe e meteo sì. «Scarica per il volo» (`PH.prefetch`) riempie le cache in anticipo: rilievo a zoom 7–11 in raggi da 450 a 60 km, tessere della base a zoom 8–13 in raggi da 60 a 6 km, con 4 scarichi in parallelo.

6.2 Carta VFR open flightmaps

La carta VFR d'Italia di open flightmaps (pacchetto MBTiles ufficiale, circa 105 MB, zoom 4–11, licenza OFMA con citazione) si scarica una volta e resta sul telefono. `carta.js` sceglie la strada: plugin nativo `Carta` nell'app Android, worker `carta-web.js` nel browser (se ci sono `Worker` e `navigator.storage.getDirectory`).

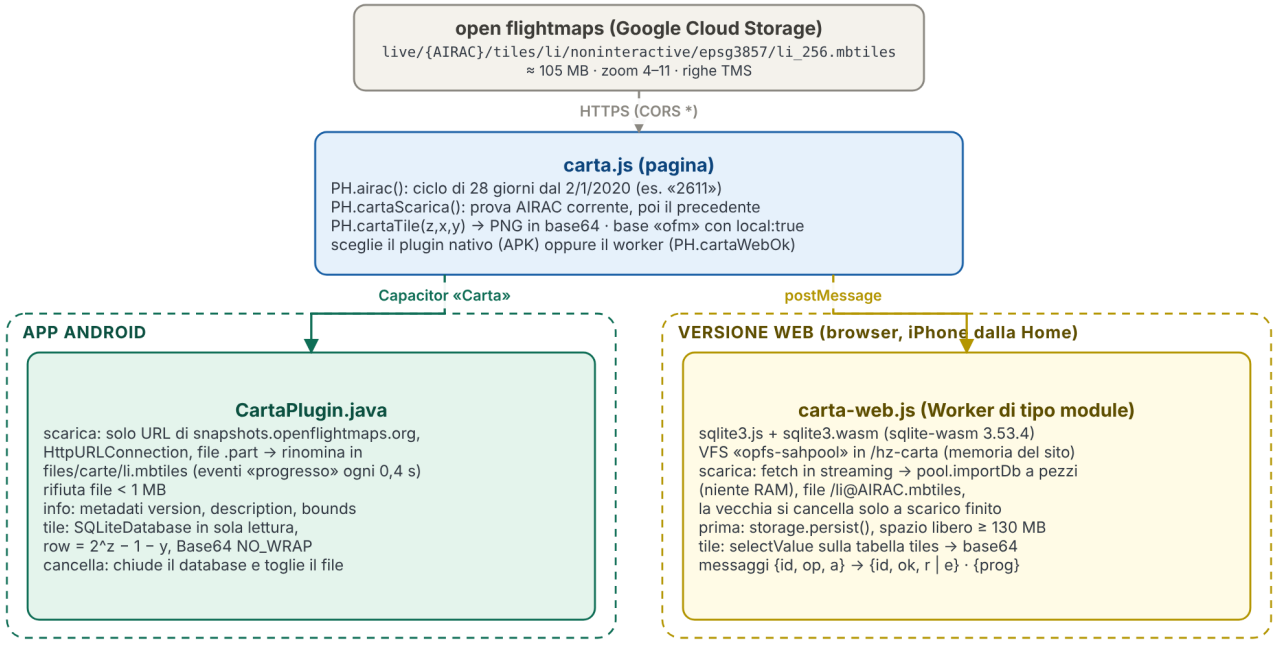


Figura 16 — Carta VFR: lo stesso pacchetto MBTiles letto dal plugin nativo (SQLite di Android) o dal worker web (sqlite-wasm con il VFS «opfs-sahpool»).

Il plugin accetta solo indirizzi di `snapshots.openflightmaps.org` e nomi `[a-z0-9_]{1,24}`; il worker scrive la carta nuova accanto alla vecchia (`/li@AIRAC.mbtiles`) e cancella la vecchia solo a scarico completo e verificato. Sul web la libreria SQLite si carica solo se `ia517p_carta_web` dice che la carta c'è; su iPhone serve la schermata Home, altrimenti iOS può cancellare i dati del sito dopo alcune settimane di inattività (l'app lo scrive).

© 2026 Enrico D'Amico · Tutti i diritti riservati

6.3 Ciclo AIRAC e fonti degli spazi aerei

- `PH.airac()` (carta.js) conta cicli di 28 giorni dal 2 gennaio 2020 (ciclo 2001) e dà la sigla AAkk; lo scarico prova il ciclo in corso e poi il precedente. Il Piano ha una sua funzione `airac` equivalente per decidere quando riscaricare gli spazi aerei.
- Fonte principale: export OpenAir giornaliero di openAIP (CC BY-NC 4.0). Riserva: open flightmaps (`tools/ofm_openair.py` nel workflow notturno → `dati/ofm_it.txt`), unita sul telefono e mai pubblicata unita (licenze diverse). `tools/confronta_spazi.py` confronta ogni notte le due fonti (`dati/confronto_spazi.md` e riepilogo della corsa).
- Quote OpenAir: «3500MSL» è in piedi; SFC/GND e AGL si riferiscono al terreno sotto il punto (`phDemAt`).

© 2026 Enrico D'Amico · Tutti i diritti riservati

7. Moduli nativi Android (Java)

Tutti i plugin estendono `com.getcapacitor.Plugin` e si registrano in `MainActivity`. Le chiamate dalla pagina sono asincrone (`PluginCall`); i dati continui arrivano con `notifyListeners` come eventi.

Plugin (nome JS)	Metodi	Eventi	Cosa fa
GloSerial	`paired`, `connect({address})`, `disconnect`	«nmea» {lines}, «state»	GPS Bluetooth classico (SPP/RFCOMM), lettura NMEA in un thread; permesso BLUETOOTH_CONNECT
Compass	`start`, `stop`	«rot» {R[9], acc}	Sensore di rotazione di Android (≤ 20 Hz), sospeso in pausa
Barometro	`info`, `start`, `stop`	«p» {hPa, acc}	Sensore di pressione di Android (`TYPE_PRESSURE`, ≈ 5 Hz), sospeso in pausa; rifiuta se il telefono non ce l'ha
Gdl90	`start({porta})`, `stop`, `status`	«traffico» {proprio, addr, lat, lon, altFt, kt, fpm, trk, cs…}	Ricevitore UDP con MulticastLock e `Gdl90Parser` (senza dipendenze da Android)
StatoTelefono	`get`	—	Batteria, rete, segnale 2G–5G, satelliti GNSS
Carta	`scarica({url,nome})`, `info`, `tile({nome,z,x,y})`, `cancella`	«progresso» {done,total}	MBTiles della carta VFR in `files/carte/`
Licenza	`info`, `billingStart`, `billingBuy`, `billingRestore`, `caffeStart`, `caffeBuy`	—	ANDROID_ID, prima installazione, installatore, firma dell'app confrontata con due impronte SHA-256; Play Billing (abbonamento rinviato, «caffe» a consumo)
Updater	`apkDownload`, `apkInstall`, `playCheck`, `playStart`	«progress»	Scarica l'APK (solo https, anche nei rimandi, al più 6) nella cache e apre l'installazione con FileProvider; Play In-App Updates
Garmin (solo H2)	`avvia`, `invia({msg})`, `apri`, `info`	«dati», «stato»	Connect IQ Mobile SDK 2.4.0, collegamento WIRELESS tramite Garmin Connect

- **Gdl90Parser** è una classe Java pura: il commento dice che «si prova anche sul PC», ma nel repository non c'è ancora un test che lo faccia (capitolo 13).
- **Licenza**: il controllo della firma considera valida l'app se il certificato coincide con una delle impronte (Play App Signing o chiave di caricamento, usata anche per gli APK di GitHub) oppure se arriva dal Play Store.
- **Updater**: il file scaricato va sempre in `cache/aggiornamento/HORIZON.apk` ed è l'unico che `apkInstall` può aprire; il permesso `REQUEST_INSTALL_PACKAGES` esiste solo nell'APK di GitHub.
- **Sicurezza di rete**: `network_security_config.xml` permette http solo verso `wms.pcn.minambiente.it`; `allowBackup=false` e `data_extraction_rules.xml` escludono i dati dai backup; FileProvider solo sulla cache.

7.1 La variante HORIZON 2 (-Ph2)

`android/app/build.gradle` legge `project.hasProperty('h2')`. Con `./gradlew assembleRelease -Ph2`: `applicationId` diventa `it.amicoen.horizon2` (si installa accanto a HORIZON), i sorgenti includono `src/h2/java` (dove c'è `GarminPlugin.java`) e si aggiunge la dipendenza `com.garmin.connectiq:ciq-companion-app-sdk:2.4.0@aar`. Il workflow cambia anche il nome dell'app in «HORIZON 2» e scrive `version.js` con versione 2.0.N e canale `h2`. Senza `-Ph2` la compilazione non contiene codice né librerie Garmin; `MainActivity` cerca la classe con `Class.forName` e, se non c'è, va avanti.

Firma: tutte le varianti si firmano con la stessa chiave (`keystore/strumento.keystore.enc` decifrata in CI); `versionCode` = numero della corsa del workflow, `minSdk` 34 (Android 14), `targetSdk` 36.

© 2026 Enrico D'Amico · Tutti i diritti riservati

8. Orologio Garmin (Connect IQ)

L'app «HORIZON Link» (`garmin/`, Monkey C, `watch-app`, API minima 3.2.0) gira su fenix 7, 7S, 7X, le versioni Pro ed epix 2, con i permessi Positioning, Sensor, Communications e SensorHistory. Parla con HORIZON 2 attraverso l'app Garmin Connect del telefono; l'identificativo dell'app (`5d505a6e…`) è lo stesso nel manifest e in `GarminPlugin.APP_ID`.

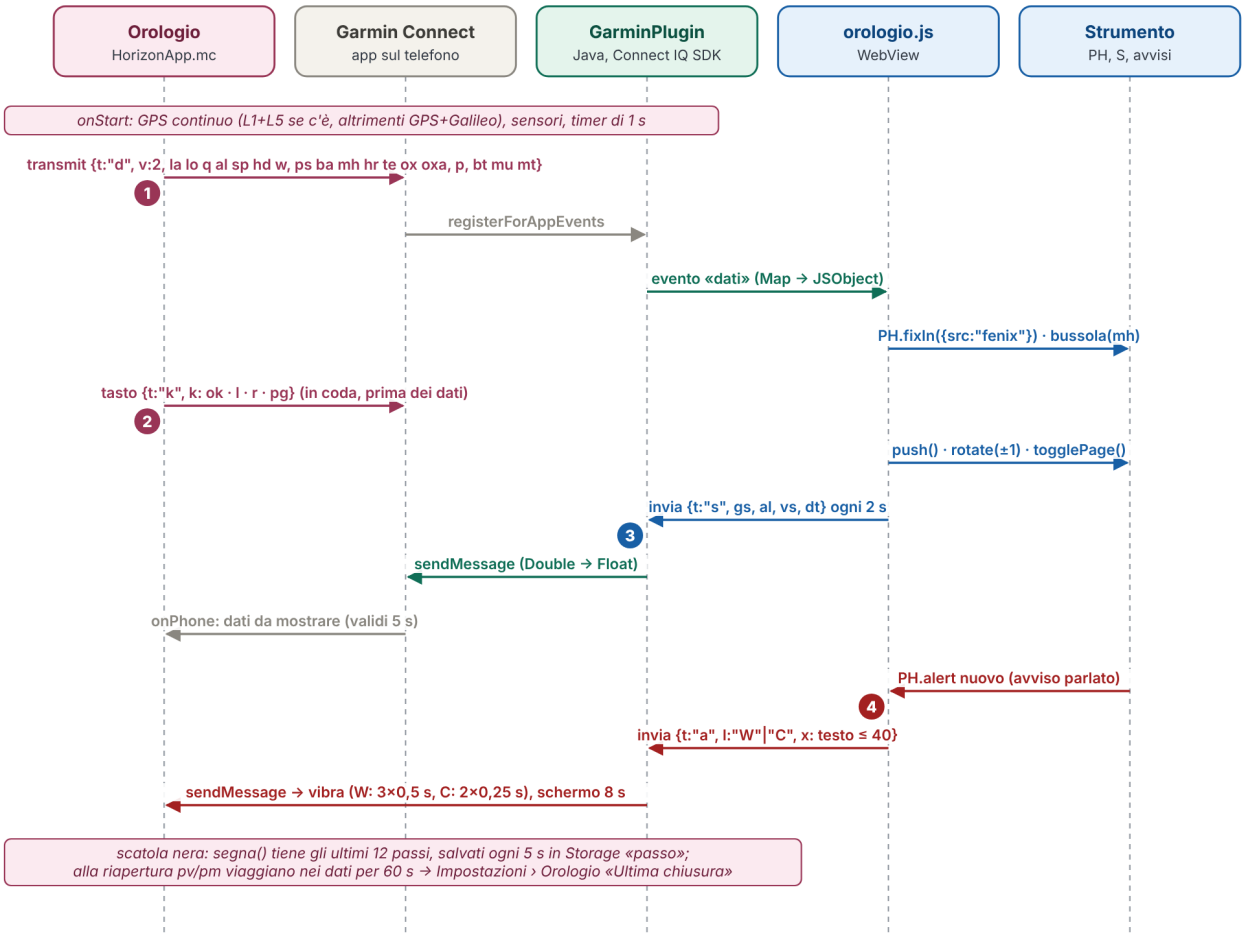


Figura 17 — Messaggi tra orologio e telefono: dati dei sensori ≈ 1 al secondo, tasti, dati da mostrare ogni 2 s, avvisi che fanno vibrare; scatola nera alla riapertura.

8.1 Dall'orologio al telefono

Chiave	Contenuto	Origine sull'orologio
`t, v`	«d» = dati, versione 2 (oppure «k» = tasto, con `k`)	—
`la, lo, q, al, sp, hd, w`	lat/lon in gradi, qualità del fix (0–4), quota GPS (m), velocità (m/s), traccia (°), ora del fix	`Position.Info` (solo con qualità ≥ POOR)
`ps, ba, mh, hr, te`	pressione al livello del mare (Pa), quota barometrica, prua magnetica (°), battito, temperatura	`Sensor.Info`
`ox, oxa`	SpO_2 (%) e età della misura (s; 0 se dal sensore)	sensore o `SensorHistory.getOxygenSaturationHistory` ogni 60 s
`p`	pressione ambiente vera (Pa): base della quota con il QNH	`Activity.getActivityInfo().ambientPressure`
`bt, mu, mt`	batteria (%), memoria usata e totale (byte)	`System.getSystemStats()`
`pv, pm`	ultimo passo e memoria della sessione precedente (scatola nera), nei primi 60 s	`Application.Storage`

- GPS: multibanda `CONFIGURATION_GPS_GLONASS_GALILEO_BEIDOU_L1_L5` se l'orologio la supporta, altrimenti GPS + Galileo, altrimenti la modalità continua di base.
- Invio (`tick` ogni 1 s → `send`): un solo messaggio in volo alla volta (`busy`); senza risposta per 15 s si riprova; la coda tiene al massimo 4 messaggi e i tasti passano prima dei dati. Il collegamento si considera perso dopo

© 2026 Enrico D'Amico · Tutti i diritti riservati

8 s senza scambi.

- Commento del sorgente: niente sensori esterni né Pulse Ox continuo «meno carico, meno chiusure»: la SpO_2 viene dallo storico (Pulse Ox «tutto il giorno» acceso sul fenix).

8.2 Dal telefono all'orologio, tasti, avvisi

- Ogni 2 s, se l'orologio è collegato, `orologio.js` manda `{t:"s", gs, al, vs, dt}` (velocità km/h, quota ft, vario m/s, destinazione con distanza e rotta). Il commento nel codice dice «ogni 2 s: il collegamento passa da Garmin Connect, non va intasato».
- `PH.alert` è avvolta: quando un avviso parla davvero (non solo si ripresenta) e «Avvisi sull'orologio» è acceso, parte `{t:"a", l:"W"|"C", x: testo di al più 40 caratteri}`. L'orologio vibra (W: tre impulsi da 0,5 s al 100%; C: due da 0,25 s al 70%), accende lo schermo e riempie il quadrante di rosso o giallo per 8 s.
- Schermata normale: stato del collegamento e del GPS, velocità e quota (dallo strumento se arrivate negli ultimi 5 s, altrimenti dai sensori dell'orologio), vario, destinazione, battito e SpO_2 (gialla sotto 92%, rossa sotto 88%; solo colore, nessun avviso).
- Tasti: START = OK (`push`), SU = ruota a sinistra, GIÙ = ruota a destra, SU tenuto (menu) = cambia pagina, GIÙ + INDIETRO insieme = cambia pagina, INDIETRO da solo = esci. GIÙ e INDIETRO aspettano 0,35 s per riconoscere la combinazione.
- Bussola dell'orologio: facoltativa e spenta di serie, perché al polso si muove col braccio; se accesa sostituisce `PH.magFromR` finché i dati sono freschi.

8.3 Scatola nera e diagnosi delle chiusure

`segna(passo)` tiene in memoria gli ultimi 12 punti attraversati dal codice («secondo», «invio al telefono», «disegno»…); ogni 5 tick la traccia si salva in `Application.Storage` alla chiave «passo», ogni 10 tick si salva la memoria usata. Se l'app si chiude per un errore (sull'orologio compare «IQ!»), alla riapertura i campi `pv`/`pm` viaggiano nei dati per un minuto e compaiono in Impostazioni › Orologio Garmin come «Ultima chiusura dell'app sull'orologio durante: …», da mandare all'autore. `onUpdate` è protetto da `try/catch`; `tick` e `onPhone` no (vedi capitolo 13).

8.4 Compilazione (horizon2.yml)

- Job «orologio»: decifra la chiave da sviluppatore `garmin/devkey.der.enc` con `KEYSTORE_PASSWORD` (alla prima corsa la crea, la cifra e la salva sul ramo), scrive la versione 1.0.N nel manifest, compila con `monkeyc` dentro l'immagine pubblica `ghcr.io/matco/connectiq-tester` un `.prg` per ogni orologio presente nel kit e il pacchetto `.iq` per lo store; poi cancella la chiave in chiaro.
- Job «telefono»: Node 22, Java 21, `npx cap sync`, firma, `gradlew assembleRelease -Ph2` → `HORIZON2-2.0.N.apk`.
- Job «pubblica» (solo su `main`): Release pre-release `horizon2-N` con APK, `.prg` e `.iq`. Sui rami `claude/**` restano solo gli artifact della corsa.

© 2026 Enrico D'Amico · Tutti i diritti riservati

9. Licenze

> Stato attuale: la verifica del codice funziona, ma **la licenza non blocca niente** (commento in `riservato.js`: «Per ora non blocca niente»). Lo schema deciso dall'autore (30 giorni di prova, poi niente uso dal vivo senza licenza, mai blocchi in volo) è descritto in `SKILL.md` ma non è ancora nel codice. Anche l'abbonamento di `licenza.js` è spento (`abbonamento:false`).

9.1 Formato del codice

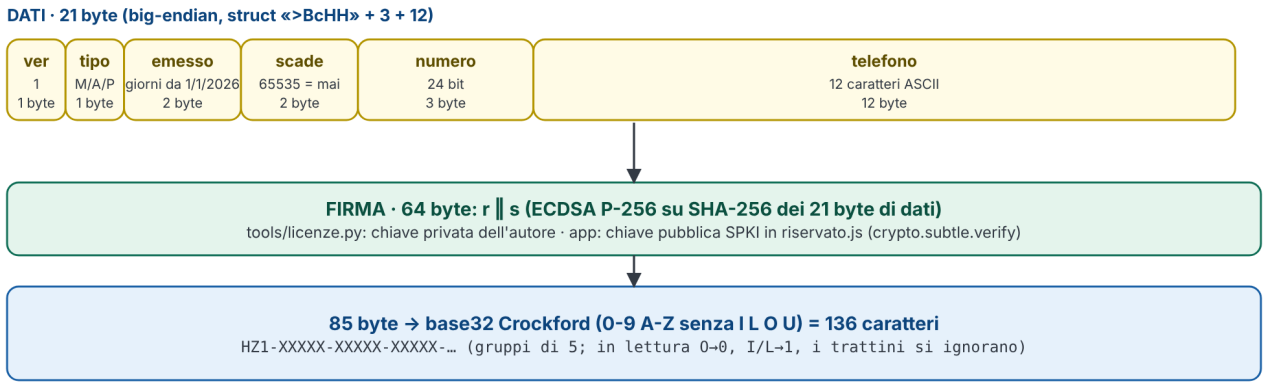


Figura 18 — Formato del codice di licenza: 21 byte di dati più 64 byte di firma ECDSA P-256, scritti in base32 Crockford con il prefisso HZ1 a gruppi di 5.

- Giorni contati dal 1° gennaio 2026; mensile = emissione + 31 giorni, annuale = + 366, perenne = 65535 (nessuna scadenza).
- Il «codice del telefono» (12 caratteri dell'alfabeto `ABCDEFGHJKLMNPQRSTUVWXYZ23456789`, mostrato come XXXX-XXXX-XXXX) è calcolato da `licenza.js`: SHA-256 di `"ia517|" + id`, con `id` = ANDROID_ID nell'app (resta uguale reinstallando) oppure un UUID casuale salvato in `localStorage` e IndexedDB nel browser.
- La firma è (r, s) di 32 byte ciascuno; la chiave pubblica (SPKI in base64) è la costante `PUB` di `riservato.js` ed è pubblica per definizione.

9.2 Verifica sul telefono, revoca, menu riservato

- Menu riservato: 5 tocchi entro 3 s sul logo in Info (`PH.riservatoOpen`). Mostra il codice del telefono, la richiesta della licenza (email del pilota + «Invia la richiesta» via Web3Forms, oppure «Scrivi dalla tua posta» con `mailto:`), il campo del codice e lo stato. Nessun prezzo né link di pagamento nell'app, in nessuna versione (regole di Google Play).
- `leggi(codice)`: toglie «HZ1», decodifica il base32 (O → 0, I/L → 1), verifica con `crypto.subtle.verify({name:"ECDSA", hash:"SHA-256"})`, poi controlla versione 1 e tipo M/A/P. Funziona senza internet.
- `codiceStato()`: «attesa» se il codice del telefono non è ancora pronto (mai «attivo» alla cieca), «altro telefono», «revocato» (numero nella lista locale), «scaduto» (con un giorno di tolleranza e l'ora più avanti mai vista, contro l'orologio portato indietro), altrimenti «attivo».
- Revoche: `revocate.json` sul ramo `licenze`, letto da `raw.githubusercontent.com` 15 s dopo l'avvio (al più una volta al giorno, `ia517p_revoche`) e ogni volta che si apre il menu riservato.

© 2026 Enrico D'Amico · Tutti i diritti riservati

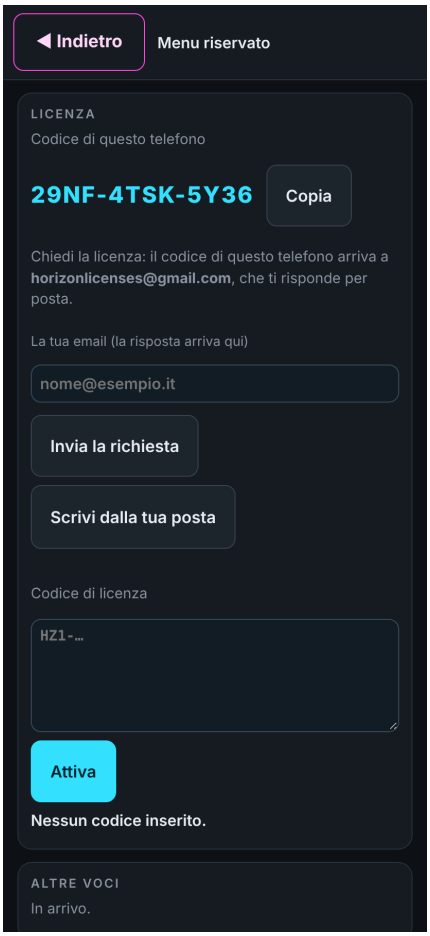


Figura 19 — Menu riservato (codice del telefono, richiesta, campo del codice)

9.3 Automazione per posta

Tutto il giro passa dalla casella Gmail di HORIZON; la chiave privata e il registro non escono mai da GitHub e sono cifrati sul ramo `licenze` (AES-256-CBC, PBKDF2 200.000 iterazioni, con `LICENZE_PASSWORD`; prima del cambio con `KEYSTORE_PASSWORD`, la funzione `scegli` legge `licenze/cifratura-v2.txt` per decidere quale).

© 2026 Enrico D'Amico · Tutti i diritti riservati

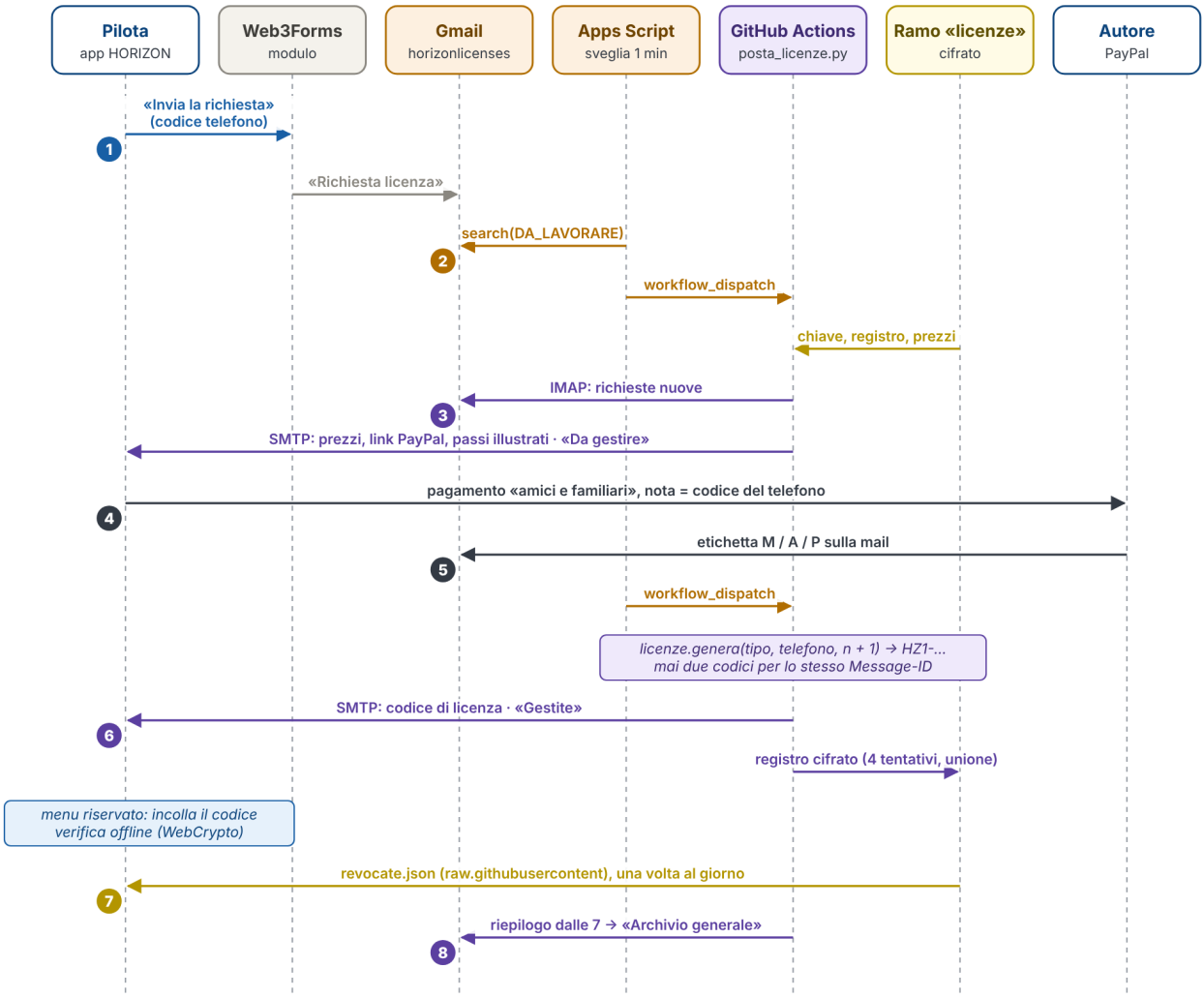


Figura 20 — Flusso completo: richiesta dall'app, Web3Forms, Gmail, sveglia di Apps Script, workflow con posta_licenze.py, pagamento, etichetta dell'autore, codice, revoche e riepilogo.

1. **Richiesta**: dall'app con Web3Forms (oggetto «HORIZON · Richiesta licenza · … telefono XXXX-…», `replyto` = pilota) oppure con la posta del pilota («HORIZON licenza XXXX-…»).
2. **Sveglia** (`HORIZON-Licenze.gs`, trigger ogni minuto): se `GmailApp.search` trova richieste nuove o mail con etichetta M/A/P non gestite, chiama `workflow_dispatch` di `licenze-posta.yml` (token fine-grained con «Actions: read/write» nella proprietà `GITHUB_TOKEN`), con una pausa di 4 minuti tra due avvii; una volta al giorno dalle 7 anche senza lavoro, per il riepilogo. Motivo: su questo repository i cron di GitHub partono con ore di ritardo (il `*/10` non è mai partito); il cron orario `23 * * * *` resta come riserva.
3. **Prezzi** (`posta_licenze.py`, passo 1): per ogni richiesta degli ultimi 30 giorni senza etichette, legge il codice del telefono dall'oggetto o dal testo; le risposte dei piloti («Re:») restano in arrivo con «Da gestire»; altrimenti risponde in testo e HTML con i tre prezzi, i link `paypal.me/…/5EUR·30EUR·150EUR` e l'immagine dei passi (`docs/licenze/paypal-passi.png`, inline con `cid:`) e sposta la mail in «Da gestire». Limiti: una mail di prezzi per telefono ogni 7 giorni, una per indirizzo al giorno, al massimo 30 al giorno (stato cifrato `licenze/prezzi.json.enc`, indirizzi solo come impronta SHA-256).
4. **Pagamento**: il pilota paga su PayPal con la nota = codice del telefono; l'autore controlla la notifica e mette alla mail l'etichetta M, A o P.
5. **Codice** (passo 2): numero = massimo del registro + 1, `licenze.genera`, verifica con la chiave pubblica, risposta al pilota con il codice e le istruzioni, riga nel registro (con nome, mail e Message-ID) scritta subito, etichetta «Gestite». Mai due codici per lo stesso Message-ID o per lo stesso telefono e tipo nello stesso giorno; senza codice del telefono la mail va in «Da vedere».
6. **Piste dei piloti** (passo 2b): dalle mail «Pista rilevata» e «Campi usati» salva solo le righe anonime `HORIZON-PISTA {…}`/`HORIZON-CAMPI {…}` in `piste/AAAA-MM-GG-<hash>.txt` sul ramo `licenze`, etichetta «Piste lette».

© 2026 Enrico D'Amico · Tutti i diritti riservati

7. **Riepilogo** (passo 3): al primo giro del giorno dopo le 7 (ora italiana) o a richiesta (input `riepilogo`), mail «HORIZON · Riepilogo licenze» con la tabella (numero, tipo, pilota, telefono, date, stato) e lo storico delle revoche, spostata (aspettando fino a 1 minuto la consegna) in «Archivio generale licenze». È l'unico elenco: il repository è pubblico e i log contengono solo conteggi.
8. **Salvataggio**: il passo «Salva il registro» gira sempre (`if: always()`), ricifra solo se il contenuto in chiaro è cambiato e, se il ramo è stato scritto da un altro giro, unisce i registri (`registro_unisci.py`) e riprova fino a 4 volte.

Azioni a mano (licenze.yml)

Azione	Cosa fa
crea-chiave	Una volta sola: coppia ECDSA P-256, privata cifrata, pubblica in chiaro nel riepilogo della corsa (va in `riservato.js`); rifiuta di ricrearla se esiste
genera	Codice per tipo e telefono (`licenze_azione.py`, riapplicabile se il push fallisce); il codice parte per posta verso la casella di HORIZON solo dopo il salvataggio del registro, mai nel riepilogo pubblico
revoca / ripristina	Aggiunge o toglie il numero in `revocate.json` con la data nello storico (solo numeri e date)
cambia-password	Ricifra tutti i `.enc` con `LICENZE_PASSWORD` (almeno 24 caratteri), verifica ogni file e scrive `cifratura-v2.txt` (sul ramo questo file esiste: il cambio è stato fatto)

I due workflow hanno gruppi di concorrenza diversi (`licenze-posta` e `licenze-manuale`), così la sveglia non annulla mai un giro manuale; il rovescio della medaglia è descritto tra le proposte (capitolo 13).

© 2026 Enrico D'Amico · Tutti i diritti riservati

10. Compilazione e pubblicazione (GitHub Actions)

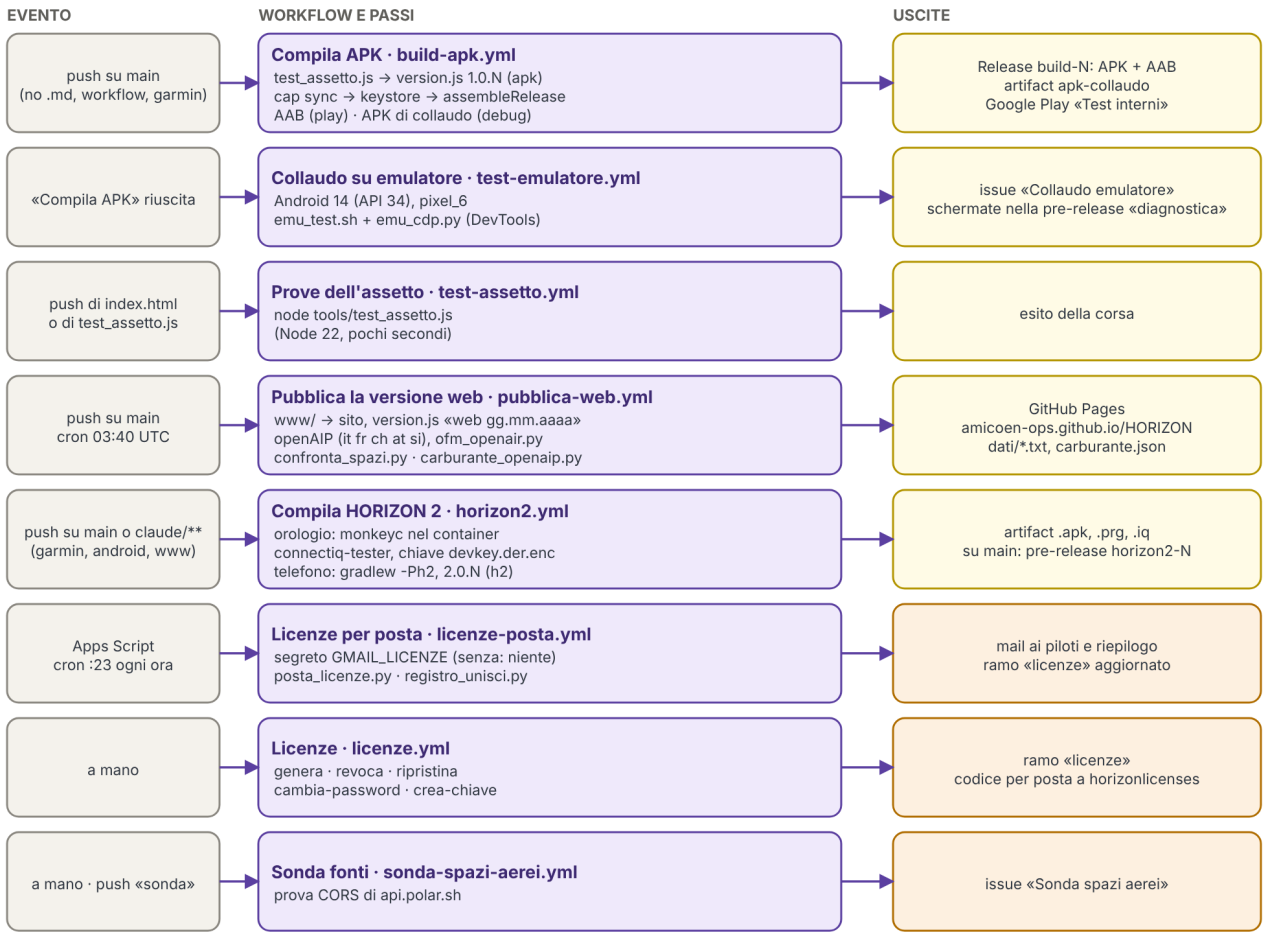


Figura 21 — Gli otto workflow: evento che li avvia, passi principali, uscite.

10.1 Numeri di versione

- APK e AAB: `1.0.<run_number>` di «Compila APK», `versionCode` = `run_number` (variabili `VERSION_NAME`, `VERSION_CODE` lette da `build.gradle`). Il tag della Release è `build-N`.
- L'app confronta il numero del tag dell'ultima Release con `IA517_BUILD.build` per proporre l'aggiornamento (APK); su Play usa il `versionCode` disponibile.
- Le novità mostrate in Info stanno nell'array `PH_CHANGES` di `index.html` (oggi la prima voce è 1.0.161) e vanno aggiornate a mano a ogni modifica visibile, insieme a `docs/play/novita/whatsnew-it-IT` (al più 500 caratteri).
- Web: `version.js` = `web gg.mm.aaaa` (data dell'ultimo commit) con `build` = primi 8 caratteri dello SHA-1 delle pagine e dei moduli: cambia solo quando cambia l'app.
- HORIZON 2: `2.0.<run_number>` di horizon2.yml; app dell'orologio `1.0.<run_number>` nel manifest.

10.2 Dettagli dei workflow

- **build-apk.yml**: parte su `main` tranne che per modifiche a soli workflow, testi `.md`, `garmin/`, `android/app/src/h2/`, `docs/garmin/`. Se le prove dell'assetto falliscono non esce nessuna APK. In caso di errore crea una issue «Compilazione N fallita» con le righe significative del log. Il caricamento su Play si salta (restando verde) se manca `PLAY_SERVICE_ACCOUNT_JSON`; `status` viene da `vars.PLAY_STATUS` (di serie «completed»).
- **test-emulatore.yml**: dopo una compilazione riuscita scarica l'artifact `apk-collaudo` di quella corsa, avvia un emulatore Android 14 con KVM, installa, prova il primo avvio senza permessi, dà la posizione finta (Carolina, 45,2683 N 7,8656 E) e pilota la WebView via DevTools (capitolo 11); schermate e logcat nella pre-release «diagnostica», sintesi in una issue.

© 2026 Enrico D'Amico · Tutti i diritti riservati

- **pubblica-web.yml**: copia `www/` in `sito/`, scrive `version.js`, scarica gli spazi aerei openAIP di it/fr/ch/at/si, la riserva open flightmaps, il confronto e i campi con carburante; aggiunge `/v2/` che rimanda alla versione attuale; pubblica con `deploy-pages`. Concorrenza «pages» con annullamento della corsa precedente.
- **sonda-spazi-aerei.yml**: nonostante il nome, prova le risposte CORS di `api.polar.sh` (per la licenza Polar rinviata) e scrive l'esito in una issue.

10.3 Segreti usati dai workflow

Segreto (solo nome)	Usato da	A cosa serve
`KEYSTORE_PASSWORD`	build-apk, horizon2, licenze, licenze-posta	Decifra la chiave di firma e la chiave Garmin; ex password delle licenze (ancora letta prima del cambio)
`LICENZE_PASSWORD`	licenze, licenze-posta	Cifratura di chiave privata, registro e stato delle licenze
`GMAIL_LICENZE`	licenze, licenze-posta	Password per le app della casella di HORIZON (IMAP/SMTP)
`PLAY_SERVICE_ACCOUNT_JSON`	build-apk	Caricamento dell'AAB in «Test interni»
`GITHUB_TOKEN (proprietà Apps Script)`	HORIZON-Licenze.gs	Token fine-grained per `workflow_dispatch` (non è un segreto di Actions)

© 2026 Enrico D'Amico · Tutti i diritti riservati

11. Prove e qualità

11.1 Banco di prova dell'assetto (tools/test_assetto.js)

Il test estrae da `index.html` il codice vero (`PhMahony`, `PH.toBody`, il blocco da `PH.GPERM` a `PH.gyroTxt`, `PH.onMotion`) e lo esegue in Node con le variabili globali sostituite. Genera dati di sensori da un moto noto (60 Hz per 10 s, con la virata tra 2 e 7 s) e misura l'errore massimo di rollio, beccheggio e accelerazione verticale. Esce con 1 se una prova non passa.

Gruppo	Casi	Soglie
Macchina in curva (non s'inclina)	destra e sinistra 20°/s con giroscopio, con la virata dal GPS, schermo a 90° e 270°, telefono inclinato 30° e livellato	rollio e beccheggio ≤ 3°, acc. verticale ≤ 0,6 m/s²
Sul tavolo	girato ±60° attorno alla verticale: assi standard, assi beta/gamma scambiati, assi scambiati ma salvati come «riconosciuti»	come sopra
Aereo in virata coordinata	30° a destra e sinistra, con giroscopio e con la virata dal GPS	rollio atteso ±3°, beccheggio ≤ 3°
Segno del giroscopio	10 serie di 1 minuto di rotazione con rumore e accelerometro arrotondato a 0,1 m/s² (S23 FE)	nessuna inversione del segno
Registrazioni vere	ogni file di `tools/registrazioni`, anche riportato agli assi standard e partendo da «αβγ invertiti»	dopo 9 s entro 8° di beccheggio e 15° di rollio dall'accelerometro mediato

Esecuzione in questo ambiente: tutte le 21 righe di prova passano (`Tutte le prove dell'assetto passano.`). Il test gira in `build-apk.yml` (blocca l'APK) e in `test-assetto.yml` a ogni push che tocca `index.html` o il test.

11.2 Registrazioni dei sensori

`tools/registrazioni/` contiene due registrazioni di un Samsung S23 FE (HORIZON 1.0.75 e 1.0.76, Android 16, WebView 156): «giri su sé stessi» e «giri lenti», con il giroscopio ad assi scambiati. Ogni file ha l'intestazione `# montaggio [...]` (la matrice usata dal telefono) e le righe nel formato di `PH.recPush`. Sono i dati che hanno portato al riconoscimento automatico dell'ordine degli assi.

11.3 Collaudo sull'emulatore

- `tools/emu_test.sh`: installa l'APK di collaudo, primo avvio senza permessi (schermata), tocco su «Consenti» se il sistema lo chiede, permessi di posizione concessi da adb, posizione finta ripetuta ogni 2 s, poi `emu_cdp.py`; alla fine estrae dal logcat crash Java, problemi del renderer, errori della console e del processo.
- `tools/emu_cdp.py`: trova il socket `webview_devtools_remote_*`, valuta espressioni nella pagina e fa schermate: avvio e versione, accettazione delle condizioni, fix GPS, «Prima del volo», Impostazioni, Piano, Info, pagine e zoom, spazi aerei, carburante, rotta, profilo, traffico, volo dimostrativo, telefono in orizzontale; misura anche il blocco più lungo del disegno.
- Limiti: l'emulatore non ha sensori di movimento reali, Bluetooth, GDL90 né orologio; la parte Java non è coperta da test unitari.

11.4 Archivio degli scarti dei sensori

`docs/sensori/scarti.json` raccoglie le misure vere dei sensori di pressione: dispositivo (solo il modello), pressione letta, riferimento (METAR o altro dispositivo), scarto in hPa e in metri, note e osservazioni (per esempio l'«avviso temporale» del fenix nelle salite e discese rapide). Serve da riferimento e, in futuro, come scarto di partenza per i dispositivi dello stesso modello. Prime misure: Galaxy S23 FE, Galaxy S23, fenix 7.

11.5 Altre verifiche

- `SKILL.md` chiede, prima di dire «provato», la compilazione con `vm.Script` di ogni `<script>` inline, il test dell'assetto e una prova nel browser con Playwright (errori di pagina e di console).
- Nessun test automatico per: verifica dei codici di licenza tra Python e WebCrypto, `posta_licenze.py`, `Gdl90Parser`, lettura NMEA, TAWS, parser OpenAir e NOTAM, Calcola rotta, service worker, app dell'orologio.

© 2026 Enrico D'Amico · Tutti i diritti riservati

12. Sicurezza e privacy

12.1 Chiavi e segreti

- **Firma delle app**: `keystore/strumento.keystore.enc` è nel repository solo cifrato (`openssl enc -aes-256-cbc -pbkdf2 -iter 200000`); la password è `KEYSTORE_PASSWORD`, usata anche come password del keystore e della chiave in `build.gradle`. La stessa password cifra la chiave da sviluppatore Garmin. Su Play la firma finale è di Google (Play App Signing); la chiave del repository è la chiave di caricamento e firma gli APK di GitHub.
- **Licenze**: chiave privata, registro e stato dei prezzi cifrati sul ramo `licenze` con la password dedicata `LICENZE_PASSWORD` (dal cambio registrato in `cifratura-v2.txt`); decifrati solo in `$RUNNER_TEMP` con `umask 077` e cancellati a fine corsa.
- **Gmail**: password per le app nel segreto `GMAIL_LICENZE`; senza il segreto il workflow non fa niente. **Apps Script**: token GitHub limitato al solo repository e ad Actions, con promemoria della scadenza nel Calendar.
- **Log**: il repository è pubblico, quindi `posta_licenze.py` stampa solo conteggi e, in caso di eccezione, solo il tipo dell'errore («potrebbe contenere un indirizzo»).

12.2 Cosa è pubblico

- Tutto il codice (con licenza «tutti i diritti riservati»), i workflow, la chiave pubblica delle licenze (`PUB`), la chiave pubblica di licenza della Play Console e le impronte dei certificati in `LicenzaPlugin.java`, la chiave pubblica di Web3Forms (`PH_INFO.w3f`, fatta per stare nelle pagine), l'indirizzo della casella di HORIZON e il link PayPal dell'autore.
- Sul ramo `licenze`: `revocate.json` (numeri e date), `chiave_pubblica.txt`, i file cifrati, `ultimo_riepilogo.txt` (una data), le righe anonime delle piste.
- I campi di input di `licenze.yml` finiscono nei log di GitHub: il workflow avverte di non mettere dati personali; il codice di licenza può comparire perché vale solo sul telefono per cui è fatto.

12.3 Impronta e filigrana dell'autore

- **Impronta nel codice**: `PH_INFO.impronta` = `HZ·ED·2026·DA8A3A7ED40C` (prime 12 cifre dello SHA-256 di una frase con il nome dell'autore). Non è un commento: il codice la usa nella prima riga di `PH.diag` (quindi in ogni segnalazione), in Info sotto la licenza e nella filigrana, per cui resta anche nel codice compresso o copiato.
- **Filigrana sulle schermate**: `#hzFil`, un testo fisso in basso a sinistra su tutte le pagine di `index.html` (in `app.html` solo se aperto da solo): «HORIZON © E. D'Amico · codice del telefono · fine dell'impronta». Carattere di 8 px ridotto a metà, grigio, opacità 0,2 e trasparente ai tocchi: durante l'uso non si nota, ma si legge ingrandendo uno screenshot e dice da quale telefono viene.
- Restano le intestazioni «© 2026 Enrico D'Amico · Tutti i diritti riservati» in testa ai file. La prova più forte dell'origine è la storia pubblica di git, con le date di ogni modifica; la filigrana è un deterrente e una traccia.

12.4 Privacy (sintesi di PRIVACY.md)

- Nessun account, pubblicità o analisi. Posizione, assetto, rotte, campi, diario e impostazioni restano sul telefono (posizione usata solo con l'app aperta).
- Escono: tessere di mappe e terreno della zona mostrata, coordinate dei campi a Open-Meteo, sigle ICAO ad aviationweather.gov, riquadro arrotondato a 0,1° a Overpass e ai servizi di traffico, controllo degli aggiornamenti a GitHub, segnalazioni inviate dal pilota (con dati tecnici facoltativi, mai la posizione).
- **Archivio comune anonimo**: con il consenso, chiesto solo a terra (`PH.aTerra`, mai in volo), l'app manda le piste rilevate (centro, direzione, lunghezza, quota, campo vicino) e una volta alla settimana il conteggio di decolli e atterraggi per campo, con un codice casuale di 8 caratteri (`PH.anonId`). Una pista entra nelle mappe di tutti se la vedono almeno 3 telefoni (centro entro 300 m, direzione entro 12°) o dopo il controllo dell'autore.
- Donazioni: Play Billing su Play, PayPal fuori dall'app su APK e web.
- Aggiornata al 6 novembre 2026: non parla ancora della richiesta di licenza (email e codice del telefono, registro con nome e mail), dell'orologio Garmin (battito, SpO_2) né della lista delle revoche (vedi proposte).

© 2026 Enrico D'Amico · Tutti i diritti riservati

13. Proposte e rischi

Osservazioni nate leggendo il codice, in ordine di priorità: P1 da fare presto, P2 utile, P3 quando capita. Dove un punto non è verificato in esecuzione lo si dice.

	Proposta	Perché
P1	Prova automatica Python ↔ WebCrypto dei codici di licenza	Il formato è scritto due volte (base32 e struttura in `licenze.py` e in `riservato.js`); `SKILL.md` lo segna come «resta da fare». Un test in CI che genera codici con una chiave di prova (tutti i tipi, numeri al limite dei 24 bit, scadenza 65535) e li verifica con `PH._licLeggi` in Node (WebCrypto è in `globalThis.crypto`) eviterebbe codici rifiutati in silenzio.
P1	Pagamenti PayPal «amici e familiari»	La mail dei prezzi chiede esplicitamente «Per amici e familiari» verso un conto personale: per PayPal è un uso non previsto per la vendita di beni e servizi (rischio di limitazione del conto) e il pilota non ha la protezione acquisti. Valutare un conto Business con «beni e servizi» o un fornitore di pagamenti, almeno prima di accendere il blocco della licenza.
P1	Blocco della licenza ancora spento	Oggi un codice valido o assente non cambia niente. Prima di accenderlo servono: le regole «mai in volo» in un punto solo, la pagina a terra, la prova di 30 giorni, la tolleranza sulle revoche senza rete e i test sui casi limite (orologio indietro, telefono cambiato, ANDROID_ID diverso dopo un ripristino di fabbrica).
P2	Numeri di licenza in conflitto	`licenze-posta.yml` e `licenze.yml` hanno gruppi di concorrenza diversi e calcolano entrambi «massimo + 1»: due giri contemporanei possono dare lo stesso numero a due telefoni. `registro_unisci.py` lo segnala con «conflitto», ma le revoche sono per numero e colpirebbero tutte e due. Un gruppo di concorrenza comune solo per la scrittura del numero, o un numero assegnato dopo il push, chiuderebbe il caso.
Fatto	Etichetta «Piste lette» tra virgolette doppie (corretto il 9/11/2026)	Trovato durante la stesura di questo dossier: `posta_licenze.py` passava `'"Piste lette"'` a `etichetta()`, che aggiunge già le virgolette alle etichette con spazi. Corretto: ora si passa `Piste lette`.
P2	Ritardi dei cron di GitHub	Sul repository i cron partono con ore di ritardo; per le licenze il problema è aggirato dalla sveglia di Apps Script, che però dipende da un token annuale (promemoria già nel Calendar). La copia notturna degli spazi aerei di `pubblica-web.yml` resta soggetta al ritardo: si potrebbe farla partire dalla stessa sveglia una volta al giorno.
P2	Diagnosi delle chiusure dell'orologio («IQ!»)	La scatola nera dice dove era il codice, non l'errore. `tick()` e `onPhone()` non sono protetti da `try/catch` (solo `onUpdate` lo è): avvolgerli e salvare il messaggio dell'eccezione in Storage darebbe la causa. Utile anche leggere `CIQ_LOG` / ERA dal portale sviluppatori e registrare il picco di memoria, non solo l'ultimo valore.
P2	PRIVACY.md da aggiornare	Mancano: richiesta di licenza (email, codice del telefono; nome e mail nel registro cifrato), dati dell'orologio in HORIZON 2 (battito e SpO_2, dati sanitari), lettura della lista delle revoche e scarico della carta VFR.
P2	Lacune nei test	Senza prove automatiche: `Gdl90Parser` (si potrebbe provare con javac nella CI, come dice il suo commento), lettura NMEA e scelta della fonte, TAWS, parser OpenAir/NOTAM, Calcola rotta, posta delle licenze (con la classe `Posta` sostituita, come nella prova locale citata in `SKILL.md`), app dell'orologio (il container `connectiq-tester` può eseguire test Monkey C).
P3	Dimensione di index.html	5.192 righe e 588 KB in un solo script, con funzioni ridefinite in più punti (per esempio `PH.show`, `togglePage`, `PH.renderImp` avvolte dai moduli). Spostare gradualmente le sezioni stabili (DEM, traffico, diario, simulazione) in moduli come quelli già esistenti renderebbe più facili test e revisioni; `test_assetto.js` dipende dalla forma esatta di alcune righe di `index.html`.
P3	Libreria SQLite anche nell'APK	`prepare_www.py` esclude `sqlite3.js`, `sqlite3.wasm` e `carta-web.js` dall'APK, ma la CI fa `cap sync` direttamente su `www/` del repository: circa 1,5 MB inutili nell'app Android (lì c'è il plugin nativo).
P3	Due calcoli del ciclo AIRAC	`PH.airac` in `carta.js` e `airac` in `app.html` fanno la stessa cosa in due modi: una funzione sola eviterebbe divergenze.
P3	Nomi e percorsi	«Sonda fonti spazi aerei» prova in realtà Polar; `build_manuale_app.py` scrive in percorsi fuori dal repository (`/mnt/user-data`, `/home/claude`) e non è riproducibile da qui; la cache del service worker ha un nome fisso (`ia517-app-1.0.0`).
Fatto	Barometro del telefono (9 novembre 2026)	Plugin `BarometroPlugin` e fusione con il GPS in `PhVario`: vario dal barometro, quota assoluta dal GPS tarato; barometro dell'orologio corretto sul telefono (capitolo 3.4). Da verificare in volo contro l'altimetro di bordo.

© 2026 Enrico D'Amico · Tutti i diritti riservati

14. Appendice: il codice

Estratti scelti, letti dal repository mentre si genera il documento (i numeri di riga sono quelli del file attuale). Le righe molto lunghe vanno a capo; quelle oltre i 360 caratteri sono accorciate con «…».

Fix del GPS del telefono · `www/index.html:2514–2520`

```
2514  PH.onPos=function(p){
2515    const c=p.coords;
2516    const fx={src:'phone',lat:c.latitude,lon:c.longitude,msl:(c.altitude!=null&&c.altitude===c.altitude)?c.al
      titude-PH.geoid:null,
2517       speed:c.speed,heading:c.heading,acc:c.accuracy,t:p.timestamp||Date.now(),recv:Date.now()};
2518    PH.phoneFix=fx;
2519    PH.fixIn(fx);
2520  };
```

Il fix del browser diventa un oggetto «fix» comune a tutte le fonti: quota riportata al livello del mare con il geoide fisso, precisione in metri, ora del fix e ora di ricezione.

Scelta della fonte di posizione · `www/index.html:2525–2537`

```
2525  PH.fixes={}; PH.srcCur=null; PH.srcCand=null;
2526  PH.srcFresh=f=>!!f && Date.now()-f.recv<2500;
2527  PH.srcPick=function(){
2528    const P=(PH.srcPref==='fenix'&&!PH.fenix)?'auto':PH.srcPref; // «solo orologio» scelto in HORIZON 2 ma qui
      l'orologio non c'è
2529    if(P!=='auto') return P;
2530    const L=Object.values(PH.fixes).filter(PH.srcFresh); if(!L.length) return PH.srcCur;
2531    const a=f=>f.acc>0?f.acc:99, best=L.reduce((x,y)=>a(y)<a(x)?y:x), cur=PH.fixes[PH.srcCur];
2532    if(!PH.srcFresh(cur)){ PH.srcCur=best.src; PH.srcCand=null; return best.src; }
2533    if(best===cur || a(best)>a(cur)*0.7){ PH.srcCand=null; return PH.srcCur; }
2534    const now=Date.now(); if(!PH.srcCand || PH.srcCand.src!==best.src) PH.srcCand={src:best.src,t:now};
2535    if(now-PH.srcCand.t>=5000){ PH.srcCur=best.src; PH.srcCand=null; PH.act&&PH.act('posizione da '+best.src+'
      ±'+Math.round(a(best))+' m'); }
2536    return PH.srcCur; };
2537  PH.fixIn=function(fx){ PH.fixes[fx.src]=fx; if(PH.srcPick()===fx.src) PH.applyFix(fx); };
```

`PH.srcPick`: con una fonte fissa la restituisce sempre; in automatico considera solo i fix degli ultimi 2,5 s, cambia fonte solo dopo 5 s di vantaggio oltre il 30% (`a(best) > a(cur)*0.7`) e subito se quella in uso tace. `PH.fixIn` applica solo il fix della fonte scelta.

Garmin GLO 2: frasi NMEA · `www/index.html:3542–3553`

```
3542  PH.gloLine=function(line){
3543    const p=PH.nmeaOk(line); if(!p) return;
3544    const G=PH.glo, typ=p[0].slice(2), now=Date.now();
3545    if(typ==='GGA'){ G.gga={q:+p[6],sats:+p[7],hdop:+p[8],alt:p[9]!==''?+p[9]:null,t:now}; return; }
3546    if(typ!=='RMC' || p[2]!=='A') return;
3547    const lat=nmeaDeg(p[3],p[4]), lon=nmeaDeg(p[5],p[6]); if(lat==null||lon==null) return;
3548    G.n++; if(!G.t0||now-G.t0>2000){ G.hz=G.t0?Math.round(G.n*1000/(now-G.t0)):0; G.n=0; G.t0=now; }
3549    const gga=G.gga&&now-G.gga.t<1500&&G.gga.q>0?G.gga:null;
3550    const
      fx={src:'glo',lat,lon,msl:gga?gga.alt:null,speed:p[7]!==''?+p[7]*0.514444:null,heading:p[8]!==''?+p[8]:null,
3551       acc:gga&&gga.hdop?Math.max(2.5,gga.hdop*3):8,sats:gga?gga.sats:null,hdop:gga?gga.hdop:null,t:now,recv:n
      ow};
3552    G.fix=fx;
3553    PH.fixIn(fx);
```

Checksum, GGA memorizzata (valida 1,5 s), RMC con stato «A» trasformata in fix; la precisione è stimata da HDOP × 3 con un minimo di 2,5 m.

© 2026 Enrico D'Amico · Tutti i diritti riservati

Cuore del filtro di assetto (Mahony) · www/index.html:2122–2135

```
2122 PhMahony.prototype.update=function(gx,gy,gz,fx,fy,fz,dt){
2123   const mag=Math.hypot(fx,fy,fz), rate=Math.hypot(gx,gy,gz);
2124   const e1=(mag-1)/this.trustG, e2=rate/this.trustRate; this.trust=Math.exp(-e1*e1)*Math.exp(-e2*e2);
2125   if(mag>0.3 && mag<3){
2126     const ax=-fx/mag, ay=-fy/mag, az=-fz/mag, [vx,vy,vz]=this.down();
2127     const ex=ay*vz-az*vy, ey=az*vx-ax*vz, ez=ax*vy-ay*vx, kk=this.kp*this.trust;
2128     if(this.ki>0 && this.trust>0.05){ const M=0.035;
2129       this.ix=clamp(this.ix+this.ki*ex*dt,-M,M); this.iy=clamp(this.iy+this.ki*ey*dt,-M,M);
     this.iz=clamp(this.iz+this.ki*ez*dt,-M,M); }
2130     gx+=kk*ex+this.ix; gy+=kk*ey+this.iy; gz+=kk*ez+this.iz;
2131   } else { gx+=this.ix; gy+=this.iy; gz+=this.iz; }
2132   const h=0.5*dt, a=this.q0, b=this.q1, c=this.q2, d=this.q3;
2133   this.q0+=(-b*gx-c*gy-d*gz)*h; this.q1+=(a*gx+c*gz-d*gy)*h; this.q2+=(a*gy-b*gz+d*gx)*h;
     this.q3+=(a*gz+b*gy-c*gx)*h;
2134   const n=1/Math.hypot(this.q0,this.q1,this.q2,this.q3); this.q0*=n; this.q1*=n; this.q2*=n; this.q3*=n; };
2135 PhMahony.prototype.euler=function(){ const a=this;
```

La fiducia nell'accelerometro (`trust`) cala con la distanza da 1 g e con la rotazione; l'errore tra gravità misurata e stimata (prodotto vettoriale) corregge il giroscopio con il guadagno proporzionale e un integrale limitato; infine il quaternione si integra e si normalizza.

Compensazione della virata con la velocità GPS · www/index.html:2481–2489

```
2481   const V=(S.gpsReal && S.gs>15)?(PH.tas||S.gs/3.6):0;
2482   let fx=fb[0]-(V>0?PH.dvdt/9.80665:0), fy=fb[1]-wb[2]*V/9.80665, fz=fb[2]+wb[1]*V/9.80665;
2483   // virata dal GPS: finché l'inclinazione stimata non ha raggiunto quella vera, la compensazione fatta con
     l'assetto stimato
2484   // lascia un vettore più lungo di 1 g e il filtro lo scarterebbe. Se la forza misurata è quella attesa
     dalla virata
2485   // (√(1+(V·ω/g)²), in volo come in macchina) ne tengo solo la direzione, così l'inclinazione converge da
     sola.
2486   if(PH.turnSrc==='gps' && V>0){ const nE=Math.hypot(1,V*PH.trkRate*D2R/9.80665),
     nF=Math.hypot(fb[0],fb[1],fb[2]), nC=Math.hypot(fx,fy,fz);
2487     if(nC>0.1){ const kc=nF/nE/nC; fx*=kc; fy*=kc; fz*=kc; } }
2488   PH.ahrs.update(wb[0],wb[1],wb[2],fx,fy,fz,dt);
2489   const [ro,pi]=PH.ahrs.euler();
```

Si tolgono dalla forza specifica le accelerazioni dovute alla virata e alla variazione di velocità; con la virata dal GPS si tiene solo la direzione della forza quando il suo modulo è quello atteso, così l'inclinazione converge.

LIVELLA: la matrice di montaggio · www/index.html:2217–2232

```
2217 PH.level=function(groundPitchDeg){
2218   const a=PH.aAvg, g=Math.hypot(a[0],a[1],a[2]); if(g<7||g>12.5){ toast('TIENI FERMO IL TELEFONO E
     RIPROVA'); return false; }
2219   const dn=[-a[0]/g,-a[1]/g,-a[2]/g]; // "giù" visto dal telefono
2220   let fw=[0,0,-1]; const dd=fw[0]*dn[0]+fw[1]*dn[1]+fw[2]*dn[2];
2221   fw=[fw[0]-dd*dn[0],fw[1]-dd*dn[1],fw[2]-dd*dn[2]];
2222   if(Math.hypot(...fw)<0.3){ const e=dn[1]; fw=[-e*dn[0],1-e*dn[1],-e*dn[2]]; } // telefono quasi in piano:
     avanti = alto dello schermo
2223   const nf=Math.hypot(...fw); fw=fw.map(x=>x/nf);
2224   const th=(groundPitchDeg||0)*D2R, c=Math.cos(th), s=Math.sin(th);
2225   const fwd=[fw[0]*c-dn[0]*s, fw[1]*c-dn[1]*s, fw[2]*c-dn[2]*s], down=[dn[0]*c+fw[0]*s, dn[1]*c+fw[1]*s,
     dn[2]*c+fw[2]*s];
2226   const right=[down[1]*fwd[2]-down[2]*fwd[1], down[2]*fwd[0]-down[0]*fwd[2],
     down[0]*fwd[1]-down[1]*fwd[0]];
2227   PH.M=[fwd,right,down]; PH.mountCal=true; PH.attF=null; PH.groundPitch=groundPitchDeg||0;
2228   lsSet('ia517p_mount',PH.M); lsSet('ia517p_gpitch',PH.groundPitch);
2229   const fb=PH.toBody(a).map(x=>x/9.80665); PH.ahrs.levelFrom(fb[0],fb[1],fb[2]); // forza specifica in g
2230   toast('ASSETTO LIVELLATO'+(PH.groundPitch?` · A TERRA ${PH.groundPitch}°`:''));
2231   return true;
2232 };
```

Dalla gravità media si ricavano «giù», «avanti» (−z dello schermo proiettato) e «destra» (prodotto vettoriale); il beccheggio a terra ruota avanti e giù. Il filtro riparte già allineato.

© 2026 Enrico D'Amico · Tutti i diritti riservati

Vario: filtro complementare del 3° ordine · `www/index.html:2144–2156`

```
PhVario.prototype.predict=function(aUp,dt){
  if(!(this.h===this.h)) return;
  if(performance.now()-this.tc>3000){ this.v*=0.97; return; } // GPS fermo: niente integrazione, il vario torna a zero
  const a=clamp(aUp,-6,6)-this.ab; this.h+=this.v*dt+0.5*a*dt*dt; this.v=clamp(this.v+a*dt,-40,40); };
PhVario.prototype.correct=function(hm,dt){
  if(!isFinite(hm)) return;
  const e=hm-this.h;
  if(!isFinite(this.h) || !isFinite(this.v) || !(dt>0) || dt>3){ this.reset(hm); this.out=0; return; }
  if(Math.abs(e)>150){ this.out=(this.out||0)+1; if(this.out>=3){ this.reset(hm); this.out=0; } return; } // un fix isolato fuori misura si scarta
  this.out=0;
  const d=Math.min(dt,1);
  this.h+=Math.min(0.9,this.k1*d)*e; this.v=clamp(this.v+Math.min(0.5/d,this.k2*d)*e,-40,40);
  this.ab=clamp(this.ab-this.k3*e*d,-1.5,1.5); this.tc=performance.now(); };
```

`predict` integra l'accelerazione verticale solo con un fix recente; `correct` corregge quota, velocità e bias, scarta i salti isolati e riparte dopo un buco.

Barometro del telefono: fusione con il GPS · `www/index.html:2552–2561`

```
PH.baroIn=function(hPa){
  if(!(hPa>100 && hPa<1150)) return;
  const B=PH.baro, now=performance.now(), dt=B.t?(now-B.t)/1000:0;
  B.p=hPa; B.pa=44330*(1-Math.pow(hPa/1013.25,0.190263)); B.t=now; B.n++;
  if(!B.on || B.ofs==null || PH.demo || PH.sim) return; // scarto non ancora agganciato al GPS
  PH.vario.correct(B.pa+B.ofs,dt);
  if(PH.vario.h===PH.vario.h){ S.alt=PH.vario.h/FT2M; S.vs=clamp(PH.vario.v,-30,30); LIVE.baro=now; }
};
PH.applyFix=function(fx){
  const now=performance.now(), tFix=fx.t;
```

Ogni lettura diventa quota-pressione standard; con lo scarto agganciato corregge il filtro del vario 5 volte al secondo. Lo scarto lo tiene giusto il GPS in `PH.applyFix`.

Punto unico degli avvisi · `www/index.html:2857–2863`

```
PH.alert=function(key,level,text,every){
  const now=Date.now(), last=PH.alerts[key]||0; PH.alertSeen[key]=now;
  const m=PH.alertMute[key]; if(m && (m===true || now<m)) return;
  if(PH.alertFlag(key,level)) PH.flag(key,text); else PH.showAlert(level,text,key);
  if(now-last<(every||60000)) return;
  PH.alerts[key]=now; PH.say(text,level==='W'); PH.buzz(level==='W'?[300,100,300,100,300]:[150,80,150]);
};
```

Silenziamento per chiave, bandierina o avviso centrale, poi voce e vibrazione al più una volta per intervallo (`every`, di serie 60 s).

TAWS · `www/index.html:2874–2886`

```
PH.checkTerrain=function(){
  if(PH.demo || !PH.flight || !S.gpsReal || lost('gps')) return;
  const ac=enu(S.lat,S.lon); if(tawsInhibit(ac.E,ac.N)) return;
  const v=S.gs/3.6, tr=(S.trk!=null?S.trk:S.hdg)*D2R; let lvl=null;
  for(let t=0;t<=60;t+=5){
    const E=ac.E+Math.sin(tr)*v*t, N=ac.N+Math.cos(tr)*v*t, z=terrain(E,N); if(z!==z) continue;
    const clr=(S.alt*FT2M+S.vs*t-z)/FT2M;
    if(t<=30 && clr<100){ lvl='W'; break; }
    if(clr<300) lvl=lvl||'C';
  }
  if(lvl==='W') PH.alert('taws','W','TERRENO! CABRA',5000);
  else if(lvl==='C') PH.alert('tawsc','C','ATTENZIONE TERRENO',30000);
};
```

Proiezione lungo la traccia ogni 5 s fino a 60 s con il vario attuale: margine sotto 100 ft entro 30 s = rosso, sotto 300 ft = giallo.

© 2026 Enrico D'Amico · Tutti i diritti riservati

GloSerialPlugin: lettura in un thread ·

android/app/src/main/java/it/amicoen/horizon/GloSerialPlugin.java:125–158

```
125     private void startReader(final BluetoothSocket s) {
126         reader = new Thread(() -> {
127             String err = null;
128             try (BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream(),
      StandardCharsets.US_ASCII))) {
129                 JSArray batch = new JSArray();
130                 long last = System.currentTimeMillis();
131                 String line;
132                 while (running && (line = br.readLine()) != null) {
133                     line = line.trim();
134                     if (line.startsWith("$")) batch.put(line);
135                     long now = System.currentTimeMillis();
136                     if (batch.length() > 0 && (now - last >= 100 || batch.length() >= 40)) {
137                         JSObject o = new JSObject();
138                         o.put("lines", batch);
139                         notifyListeners("nmea", o);
140                         batch = new JSArray();
141                         last = now;
142                     }
143                 }
144             } catch (IOException e) {
145                 err = e.getMessage();
146             }
147             if (running) {
148                 running = false;
149                 JSObject st = new JSObject();
150                 st.put("connected", false);
151                 st.put("error", err != null ? err : "collegamento chiuso");
152                 notifyListeners("state", st);
153             }
154             closeQuietly();
155         }, "glo-reader");
156         reader.start();
157     }
158
```

Righe NMEA raccolte e mandate alla pagina a gruppi (100 ms o 40 righe); alla caduta del collegamento l'evento «state» avvisa la pagina, che torna al GPS del telefono.

Gdl90Parser: messaggi, escape e CRC ·

android/app/src/main/java/it/amicoen/horizon/Gdl90Parser.java:49–79

```
49      public List<Target> parse(byte[] buf, int len) {
50          List<Target> out = new ArrayList<>();
51          int i = 0;
52          while (i < len) {
53              while (i < len && buf[i] != 0x7E) i++;
54              int start = ++i;
55              while (i < len && buf[i] != 0x7E) i++;
56              if (i >= len) break;
57              int end = i; // buf[end] è il 0x7E di chiusura
58              if (end - start < 3) continue; // due 0x7E di fila: il secondo apre il messaggio successivo
59              byte[] m = new byte[end - start];
60              int n = 0;
61              for (int k = start; k < end; k++) {
62                  byte b = buf[k];
63                  if (b == 0x7D && k + 1 < end) { b = (byte) (buf[++k] ^ 0x20); }
64                  m[n++] = b;
65              }
66              frames++;
67              if (n < 3) { bad++; continue; }
68              int fcs = (m[n - 2] & 0xFF) | ((m[n - 1] & 0xFF) << 8);
69              if (crc(m, n - 2) != fcs) { bad++; continue; }
70              int id = m[0] & 0xFF;
71              if ((id == 20 || id == 10) && n - 2 >= 28) {
72                  Target t = report(m, id == 10);
73                  if (t != null) out.add(t);
74              }
75              i = end; // il 0x7E di chiusura può aprire il messaggio dopo
76          }
77          return out;
78      }
79
```

Ogni messaggio è racchiuso tra 0x7E; l'escape 0x7D si toglie con XOR 0x20; il CRC-CCITT è in coda con il byte meno significativo prima. Si leggono solo i messaggi 10 e 20 lunghi almeno 28 byte.

© 2026 Enrico D'Amico · Tutti i diritti riservati

orologio.js: messaggi dall'orologio · `www/orologio.js:50–62`

```
function dati(d){
  if(d.t==='k'){ PH.act&&PH.act('orologio tasto '+d.k);
    try{ if(d.k==='ok') push(); else if(d.k==='l') rotate(-1); else if(d.k==='r') rotate(1); else if(d.k==='pg'){ closeMenu(); togglePage(); toast(PAGE==='pfd'?'ORIZZONTE':'MAPPA'); } }catch(e){}
    return; }
  if(d.t!=='d') return;
  PH.fenix.d=d; PH.fenix.t=Date.now(); PH.fenix.n++;
  if(d.p!=null) scartoBaro(d.p);
  if(PH.fenixCfg.bussola && d.mh!=null) bussola(d.mh);
  if(d.la!=null && (d.q||0)>=2){
    const bm=PH.fenixCfg.baro&&d.p!=null?baroMsl(d.p):null;
    PH.fixIn({src:'fenix',lat:d.la,lon:d.lo,msl:bm!=null?bm:(d.al!=null?d.al:null),speed:d.sp!=null?d.sp:null,heading:d.hd!=null?d.hd:null,
       acc:QM[d.q]||60,t:Date.now(),recv:Date.now()});
  }
```

I tasti diventano i comandi della manopola; i dati diventano un fix della fonte «fenix» con precisione stimata dalla qualità Garmin e, se richiesto, quota barometrica con il QNH.

orologio.js: dati e avvisi verso l'orologio · `www/orologio.js:68–74`

```
setInterval(()=>{ if(!PH.fenix.stato.collegato) return;
  let dt=''; try{ if(NAV.dest){ const b=distBrg(S,NAV.dest); dt=`${NAV.dest.short.slice(0,10)} ${(b.d/1000).toFixed(1)} km ${String(Math.round(b.brg)).padStart(3,'0')}°`; } }catch(e){}
  G.invia({msg:{t:'s',gs:Math.round(S.gs||0),al:Math.round(S.alt||0),vs:+(S.vs||0).toFixed(1),dt}}).catch(()=>{}); },2000); // ogni 2 s: il collegamento passa da Garmin Connect, non va intasato
// avvisi: quando lo strumento parla, l'orologio vibra e lo scrive
{ const prev=PH.alert; PH.alert=function(key,level,text){ const b=PH.alerts[key]; const r=prev.apply(this,arguments);
  if(PH.alerts[key]!==b && PH.fenixCfg.avvisi && PH.fenix.stato.collegato)
  G.invia({msg:{t:'a',l:level==='W'?'W':'C',x:String(text).slice(0,40)}}).catch(()=>{});
  return r; }; }
```

Dati da mostrare ogni 2 s; `PH.alert` avvolta per mandare all'orologio solo gli avvisi che hanno appena parlato.

© 2026 Enrico D'Amico · Tutti i diritti riservati

HorizonApp.mc: dati() · garmin/source/HorizonApp.mc:236–269

```
236     function dati() {
237         segna("lettura dei sensori");
238         var d = {"t" => "d", "v" => 2};
239         if (pos != null && pos.position != null && pos.accuracy != null && pos.accuracy >=
    Position.QUALITY_POOR) {
240             var ll = pos.position.toDegrees();
241             d.put("la", ll[0]);
242             d.put("lo", ll[1]);
243             d.put("q", pos.accuracy);
244             if (pos.altitude != null) { d.put("al", pos.altitude); }
245             if (pos.speed != null) { d.put("sp", pos.speed); }
246             if (pos.heading != null) { d.put("hd", Math.toDegrees(pos.heading)); }
247             if (pos.when != null) { d.put("w", pos.when.value()); }
248         }
249         if (sens != null) {
250             if (sens.pressure != null) { d.put("ps", sens.pressure); } // pressione riportata al livello del
    mare (come un QNH)
251             if (sens.altitude != null) { d.put("ba", sens.altitude); }
252             if (sens.heading != null) { d.put("mh", Math.toDegrees(sens.heading)); }
253             if (sens.heartRate != null) { d.put("hr", sens.heartRate); }
254             if (sens.temperature != null) { d.put("te", sens.temperature); }
255             if ((sens has :oxygenSaturation) && sens.oxygenSaturation != null) { d.put("ox",
    sens.oxygenSaturation); d.put("oxa", 0); }
256         }
257         if (d.get("ox") == null && oxH != null) {
258             d.put("ox", oxH);
259             d.put("oxa", oxHT > 0 ? Time.now().value() - oxHT : -1); // età della misura in secondi
260         }
261         // pressione vera dell'aria dove si è (per la quota con il QNH dello strumento)
262         var ai = Activity.getActivityInfo();
263         if (ai != null && (ai has :ambientPressure) && ai.ambientPressure != null) { d.put("p",
    ai.ambientPressure); }
264         var st = System.getSystemStats();
265         if (st != null && st.battery != null) { d.put("bt", st.battery.toNumber()); }
266         if (st != null) { d.put("mu", st.usedMemory); d.put("mt", st.totalMemory); }
267         if (nTick < 60 && prevPasso != null) { d.put("pv", prevPasso); if (prevMem != null) { d.put("pm",
    prevMem); } }
268         return d;
269     }
```

Il dizionario dei sensori con chiavi corte per occupare poco nella trasmissione: posizione solo con qualità almeno POOR, SpO_2 dal sensore o dallo storico con l'età, pressione ambiente, memoria e la scatola nera della sessione precedente.

HorizonApp.mc: il secondo dell'orologio · garmin/source/HorizonApp.mc:173–191

```
173     function tick() as Void {
174         var now = System.getTimer();
175         // l'invio passa da Garmin Connect e può metterci parecchi secondi: mai un secondo invio finché il
    primo non risponde
176         // (troppi invii in attesa fanno chiudere l'app); dopo 15 s senza risposta si riprova
177         if (busy && now - busyT > 15000) { busy = false; }
178         if (linkOk && now - linkT > 8000) { linkOk = false; }
179         nTick++;
180         if (nTick % 5 == 0) { salvaTraccia(null); traccia = []; } // gli ultimi passi: se l'app si chiude da
    sola, restano qui
181         segna("secondo");
182         if (nTick % 10 == 0) {
183             var ms = System.getSystemStats();
184             try { Application.Storage.setValue("mem", ms.usedMemory.toString() + "/" +
    ms.totalMemory.toString()); } catch (e) { }
185         }
186         if (nTick % 60 == 5) { segna("storico SpO2"); storicoOx(); }
187         if (queue.size() > 4) { queue = queue.slice(queue.size() - 4, null); } // coda corta: i dati vecchi
    non servono
188         if (queue.size() == 0 && !busy) { queue.add(dati()); }
189         send();
190         WatchUi.requestUpdate();
191     }
```

Un solo invio alla volta (Garmin Connect può metterci secondi), coda corta, salvataggio periodico della traccia e della memoria.

© 2026 Enrico D'Amico · Tutti i diritti riservati

riservato.js: verifica del codice · `www/riservato.js:29–45`

```
async function leggi(codice){
  let c=String(codice||'').trim().toUpperCase(); if(!c) return {ok:false,perche:'nessun codice'};
  if(c.startsWith('HZ1')) c=c.slice(3); else return {ok:false,perche:'non è un codice di HORIZON (inizia con HZ1)'};
  const raw=unb32(c); if(raw.length<85) return {ok:false,perche:'codice incompleto: copialo tutto'};
  const dati=raw.slice(0,21), firma=raw.slice(21,85);
  if(/^__/.test(PUB)) return {ok:false,perche:'verifica non ancora attiva in questa versione'};
  try{ const der=Uint8Array.from(atob(PUB),ch=>ch.charCodeAt(0));
    const key=await crypto.subtle.importKey('spki',der,{name:'ECDSA',namedCurve:'P-256'},false,['verify']);
    const ok=await crypto.subtle.verify({name:'ECDSA',hash:'SHA-256'},key,firma,dati);
    if(!ok) return {ok:false,perche:'codice non valido (firma errata)'}; }
  catch(e){ return {ok:false,perche:'verifica non riuscita su questo telefono'}; }
  const dv=new DataView(dati.buffer,dati.byteOffset,21), tipo=String.fromCharCode(dati[1]),
  em=dv.getUint16(2), sc=dv.getUint16(4);
  const numero=(dati[6]<<16)|(dati[7]<<8)|dati[8], tel=String.fromCharCode(...dati.slice(9,21));
  if(dati[0]!==1) return {ok:false,perche:'versione del codice non riconosciuta: aggiorna HORIZON'};
  if(!/^[MAP]$/.test(tipo)) return {ok:false,perche:'tipo di licenza sconosciuto'};
  return {ok:true,versione:dati[0],tipo,emesso:giorno(em),scade:sc===65535?null:giorno(sc),numero,tel};
}
```

Decodifica base32, verifica ECDSA P-256 con WebCrypto sulla chiave pubblica, poi lettura dei campi con `DataView` (big-endian, come `struct.pack('>BcHH')` in Python).

licenze.py: generazione del codice · `tools/licenze.py:63–72`

```
def genera(priv_pem, tipo, tel, numero, oggi=None):
    tipo = tipo.upper()
    if tipo not in DURATA: raise SystemExit('Tipo: M (mensile), A (annuale) o P (perenne)')
    oggi = oggi or datetime.date.today()
    em = giorno(oggi); sc = NESSUNA if DURATA[tipo] is None else em + DURATA[tipo]
    dati = struct.pack('>BcHH', 1, tipo.encode(), em, sc) + int(numero).to_bytes(3, 'big') +
  telefono(tel).encode()
    key = serialization.load_pem_private_key(priv_pem, password=None)
    r, s = decode_dss_signature(key.sign(dati, ec.ECDSA(hashes.SHA256())))
    testo = b32(dati + r.to_bytes(32, 'big') + s.to_bytes(32, 'big'))
    return 'HZ1-' + '-'.join(testo[i:i + 5] for i in range(0, len(testo), 5))
```

Struttura binaria, firma DER convertita in (r, s) di 32 byte, base32 a gruppi di 5 con il prefisso HZ1.

© 2026 Enrico D'Amico · Tutti i diritti riservati

posta_licenze.py: dal pagamento al codice · `tools/posta_licenze.py:321–348`

```
    # 2. pagate: etichetta M, A o P messa dall'autore → codice, poi «Gestite» fuori dalla Posta in arrivo.
    # Mai due codici per la stessa richiesta: il registro tiene il Message-ID della mail a cui si è risposto.
    fatti = set()
    for tipo in 'MAP':
        for uid in posta.cerca(f'label:{tipo.lower()} -label:gestite -from:me'):
            try:
                msg = posta.leggi(uid)
                if msg is None: continue
                mid = (msg.get('Message-ID') or '').strip()
                tel = tel_da(dec(msg.get('Subject'))) or tel_da(testo_di(msg))
                gia = (mid and any(r.get('msgid') == mid for r in registro)) or \
                      any(r.get('telefono') == tel and r.get('tipo') == tipo and r.get('emesso') ==
oggi_d.isoformat() and r.get('origine') == 'posta' for r in registro)
                if (tipo, tel) in fatti or gia: # già mandato (anche in un giro precedente interrotto): solo
etichette
                    posta.etichetta(uid, aggiungi=[GESTITE], togli=[tipo, DA_GESTIRE, INBOX]); continue
                if not tel:
                    posta.etichetta(uid, aggiungi=[DA_VEDERE, INBOX], togli=[tipo]); conti['da vedere'] += 1;
continue
                n = max([r['numero'] for r in registro] + [0]) + 1
                codice = licenze.genera(priv, tipo, tel, n)
                info = licenze.leggi(pub, codice)
                if not posta.rispondi(msg, *testo_codice(tipo, codice, info['scade'])):
                    posta.etichetta(uid, aggiungi=[DA_VEDERE, INBOX], togli=[tipo]); conti['da vedere'] += 1;
continue
                registro.append({'numero': n, 'tipo': tipo, 'telefono': info['telefono'], 'emesso':
info['emesso'],
                                 'scade': info['scade'], 'origine': 'posta', 'nome': posta.ultimo[0], 'mail':
posta.ultimo[1], 'msgid': mid})
                json.dump(registro, open(env['REGISTRO'], 'w'), indent=1) # subito: se dopo qualcosa va
storto il numero resta usato
                fatti.add((tipo, tel)); conti['codici'] += 1
                posta.etichetta(uid, aggiungi=[GESTITE], togli=[tipo, DA_GESTIRE, INBOX])
            except Exception:
                errore(uid, tipo)
```

Ciclo sulle etichette M, A, P: controlli contro i doppioni (Message-ID nel registro, stesso telefono e tipo oggi), numero nuovo, codice verificato, risposta, registro scritto subito, etichette sistemate. Ogni mail è trattata da sola: un errore la manda in «Da vedere».

Apps Script: la sveglia · `tools/gmail-licenze/HORIZON-Licenze.gs:55–66`

```
function sveglia() {
  const p = PropertiesService.getScriptProperties(), ora = new Date();
  const oggi = Utilities.formatDate(ora, 'Europe/Rome', 'yyyy-MM-dd'), h = +Utilities.formatDate(ora,
'Europe/Rome', 'H');
  const ultimo = +(p.getProperty('ULTIMO_AVVIO') || 0);
  if (ora.getTime() - ultimo < PAUSA_MS) return;
  const serve = daLavorare_() > 0 || (h >= 7 && p.getProperty('GIRO_DEL_GIORNO') !== oggi);
  if (!serve) return;
  const err = avviaGiro_(false);
  if (err) { console.log(err); return; }
  p.setProperty('ULTIMO_AVVIO', String(ora.getTime()));
  if (p.getProperty('GIRO_DEL_GIORNO') !== oggi) { p.setProperty('GIRO_DEL_GIORNO', oggi); try {
calendario_(); } catch (e) { console.log('calendario: ' + e); } }
}
```

Ogni minuto, con 4 minuti di pausa dopo un avvio: se c'è lavoro o se non è ancora partito il giro del giorno (dalle 7), chiama `workflow_dispatch`.

© 2026 Enrico D'Amico · Tutti i diritti riservati

carta-web.js: importazione nella memoria del sito · `www/carta-web.js:51–70`

```
51    async scarica({ url, nome, versione }) {
52      await avvia();
53      const f = '/' + nome + '@' + versione + '.mbtiles';
54      const r = await fetch(url, { cache: 'no-store' });
55      if (!r.ok) throw new Error('risposta ' + r.status);
56      const tot = +r.headers.get('content-length') || 0, rd = r.body.getReader();
57      let fatti = 0, ultimo = 0;
58      if (file().length >= pool.getCapacity() - 1) await pool.addCapacity(2);
59      chiudi(f);
60      await pool.importDb(f, async () => {
61        const { done, value } = await rd.read(); if (done) return undefined;
62        fatti += value.byteLength;
63        if (fatti - ultimo > 512 * 1024) { ultimo = fatti; postMessage({ prog: [fatti, tot] }); }
64        return value;
65      });
66      if (tot && fatti !== tot) { pool.unlink(f); throw new Error('scarico incompleto'); }
67      try { apri(f).selectValue('select count(*) from metadata'); } catch (e) { chiudi(f); pool.unlink(f); throw
      new Error('file non valido'); }
68      for (const g of file()) if (g !== f && nomeDi(g) === nome) { chiudi(g); pool.unlink(g); } // le vecchie
      solo adesso
69      return { ok: true, size: fatti };
70    },
```

Il corpo della risposta passa a pezzi da `fetch` a `pool.importDb` senza tenere il file in RAM; la carta si verifica aprendo i metadati e le versioni vecchie si cancellano solo alla fine.

Service worker: strategie di cache · `www/sw.js:24–33`

```
24  self.addEventListener('fetch',e=>{
25    const req=e.request; if(req.method!=='GET') return;
26    const u=new URL(req.url);
27    if(u.searchParams.has('chk')) return; // controllo della versione: sempre dalla rete
28    if(u.hostname==='s3.amazonaws.com' && u.pathname.includes('/elevation-tiles-prod/'))
      e.respondWith(cacheFirst(req,'ia517-dem',4000));
29    else if(/tile\.opentopomap\.org$|tile\.openstreetmap\.org$/.test(u.hostname))
      e.respondWith(cacheFirst(req,'ia517-map',3000));
30    else if(u.hostname==='api.open-meteo.com') e.respondWith(networkFirst(req,'ia517-wx'));
31    else if(/fonts\.(googleapis|gstatic)\.com$/.test(u.hostname))
      e.respondWith(cacheFirst(req,'ia517-fonts',40));
32    else if(u.origin===location.origin && !NATIVE) e.respondWith(staleWhileRevalidate(req));
33  });
```

Terreno e mappe dalla cache, meteo dalla rete con ripiego, pagine del sito «stale while revalidate»; il controllo della versione (`?chk`) va sempre in rete.

Documento generato da `tools/build_dossier_tecnico.py` il 10 novembre 2026. Figure: 21; estratti di codice: 22. Diagrammi rifatti con `node docs/dossier/src/disegna.js`, schermate con `docs/dossier/src/schermate.js`.

© 2026 Enrico D'Amico · Tutti i diritti riservati